

DATA STRUCTURE AND ALGORITHMS

LAB MANUAL (23CSE203)

SUBMITTED BY	
NAME	B. SAI TRIPURA
ROLLNO	AV.SC.U4CSE25018
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-A
CREDITS	5

SUBMITTED TO	
NAME	Dr. Raj Kumar Batchu
DEPARTMENT	(SCHOOL OF COMPUTING)
DESIGNATION	ASST.PROFESSOR(SL.GRADE)

INDEX:-

S. No	*Experiment*	*Page No.*	*DATE*	*REMARKS*
1	*1(a) Recursive Rocket Launch Countdown* *1(b) Recursive compound interest growth (power calculation)* *1(c) Recursive Search for Employee ID in a List* *1(d) Recursive Factorial Calculation* *1(e) Recursive Fibonacci Series Generation*	*6-7* *8-9* *10-11* *12-13* *14-15*	*23-07-26*	
2	*2(a) linear search* *2(b) binary search*	*16-18* *19-21*	*30-07-26*	
3	*3(a) bubble sort* *3(b) insertion sort* *3(c) merge sort* *3(d) selection sort* *3(e) quick sort*	*22-24* *25-27* *28-30* *31-33* *34-35*	*06-08-26*	

S. No	*Experiment*	*Page No.*	*DATE*	*REMARKS*
4(a)	*Write a Python program to implement a Singly Linked List using a menu-driven approach. The program should:* *Create a singly linked list from an array of elements.* *Insert a node at the beginning.* *Insert a node at the end.* *Insert a node at a specific index.* *Delete a node from the beginning.* *Delete a node from the end.* *Delete a node containing a specific value.* *Count the number of nodes.* *Display the linked list.* *Exit the program.*	*36-41*	*13-08-26*	
4(b)	*Write a Python program to implement a Doubly Linked List using a menu-driven approach. The program should contain all the menu driven items listed in the above question* *1.Create a singly linked list from an array of elements.* *2.Insert a node at the beginning.* *3.Insert a node at the end.*	*42-50*	*13-08-26*	

S. No	*Experiment*	*Page No.*	*DATE*	*REMARKS*
	4.Insert a node at a specific index. *5.Delete a node from the beginning.* *6.Delete a node from the end.* *7.Delete a node containing a specific value.* *8.Count the number of nodes.* *9.Display the linked list.* *10.Exit the program.*			
4(c)	*Write a Python program to implement a Circular Linked List using a menu-driven approach. The program should contain all the menu driven items listed in the above question* *1.Create a singly linked list from an array of elements.* *2.Insert a node at the beginning.* *3.Insert a node at the end.* *4.Insert a node at a specific index.* *5.Delete a node from the beginning.* *6.Delete a node from the end.* *7.Delete a node containing a specific value.* *8.Count the number of nodes.* *9.Display the linked list.*	*51-59*	*13-08-26*	

S. No	*Experiment*	*Page No.*	*DATE*	*REMARKS*
	10.Exit the program.			
5(a) *5(b)*	Write a Python program to implement a Stack using Linked List with operations such as Push, Pop, Peek, and Display. Write a Python program to implement a Stack using Array with operations such as Push, Pop, Peek, and Display.	*60-63* *64-65*	*27-08-26*	
6(a) *6(b)*	Write a Python program to implement a Queue using Linked List with operations such as Enqueue, Dequeue, and Display.(menu driven program) Write a Python program to implement a Queue using Array with operations such as Enqueue, Dequeue, and Display. (menu driven program	*66-70* *70-71*	*03-09-26*	
7(a) *7(b)*	Write a Python program to implement a Binary Tree using Array representation with operations such as Insertion, Traversal, and Display. Write a Python program to implement a Binary Tree using Linked List representation with operations such as Insertion, Traversal, and Display.	*74-75* *75-78*	*24-09-26*	

S. No	*Experiment*	*Page No.*	*DATE*	*REMARKS*
8(a)	Write a Python program to implement a Binary Search Tree (BST) with operations such as Insertion, Searching, Deletion, and Traversal using array	*79-80*	*08-10-26*	
8(b)	Write a Python program to implement a Binary Search Tree (BST) with operations such as Insertion, Searching, Deletion, and Traversal using list	*81-82*		
9(a)	Write a Python program to implement an AVL Tree with operations such as Insertion, Deletion, and Traversal, maintaining the balance factor through rotations using array.	*83-84*	*22-10-26*	
9(b)	Write a Python program to implement an AVL Tree with operations such as Insertion, Deletion, and Traversal, maintaining the balance factor through rotations using list	*85-87*		
10	Write a Python program to implement hashing using separate chaining, linear probing, and quadratic probing.	*88-91*	*29-10-26*	

LAB-1

1(a)

Recursive Rocket Launch Countdown

Aim:-

A rocket launch system displays a countdown before launching the rocket. Instead of using a loop, implement the countdown using recursion. Write a recursive Python program to display the countdown from n to 1, followed by "Launch!".

Code:-

lab-4 rocket.py - C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\lab-4 rocket.py (3.14.6)

File Edit Format Run Options Window Help

```
n = int(input("Please enter the number of seconds you want to count: "))

def roc(n):
    for i in range(n, -1, -1):
        print(i)

    print("LAUNCH")

if n <= 0:
    print("Numbers can't be counted as the rocket has already launched.")
else:
    roc(n)
```

Output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\lab-4 rocket.py ===
Please enter the number of seconds you want to count: 10
10
9
8
7
6
5
4
3
2
1
0
LAUNCH
>>>
```

Explanation:-

The countdown function takes the current value of n as its parameter. On every call it first checks whether n has reached 0, which is the base case. When n is not 0, the function prints the current value of n and then calls itself again with the argument reduced by one, before the current call has finished. This pushes a new call on top of the previous one, so execution moves into the new call while the earlier one waits for it to complete. This repeats, with n decreasing by one each time, until the argument passed in becomes 0. At that point the base case is triggered, the message "Launch!" is printed, and the function returns without making another call. Because there is nothing left to execute after the recursive call in each earlier call, those calls simply return one after another once the base case finishes, without producing any further output. The countdown numbers therefore appear in decreasing order first, since each value is printed before the next call is made, and "Launch!" appears last, once the base case at the bottom of the recursion has been reached.

Error Table:-

S.No	*Error / Exception*	*Cause and Correction*
1	*Indentation error*	*Corrected it by giving required spaces*
2	*----------*	*----------*
3	*----------*	*----------*

1(b)

Recursive Compound Interest Growth (Power Calculation)

Aim:-

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive Python program to calculate P^n (power), where P is the principal growth factor and n is the number of years.

Code:-

bank.py - C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\bank.py (3.14.6)

File Edit Format Run Options Window Help

```
# Recursive function to calculate p^n (power)
def power(p, n):
    # Base case: anything raised to power 0 is 1
    if n == 0:
        return 1
    # Recursive case: p^n = p * p^(n-1)
    else:
        return p * power(p, n - 1)


# Main program to calculate Compound Interest
def compound_interest():
    principal = float(input("Enter the Principal amount: "))
    rate = float(input("Enter the Rate of Interest (in %): "))
    years = int(input("Enter the Number of Years: "))

    # Growth factor per year = (1 + rate/100)
    growth_factor = 1 + (rate / 100)

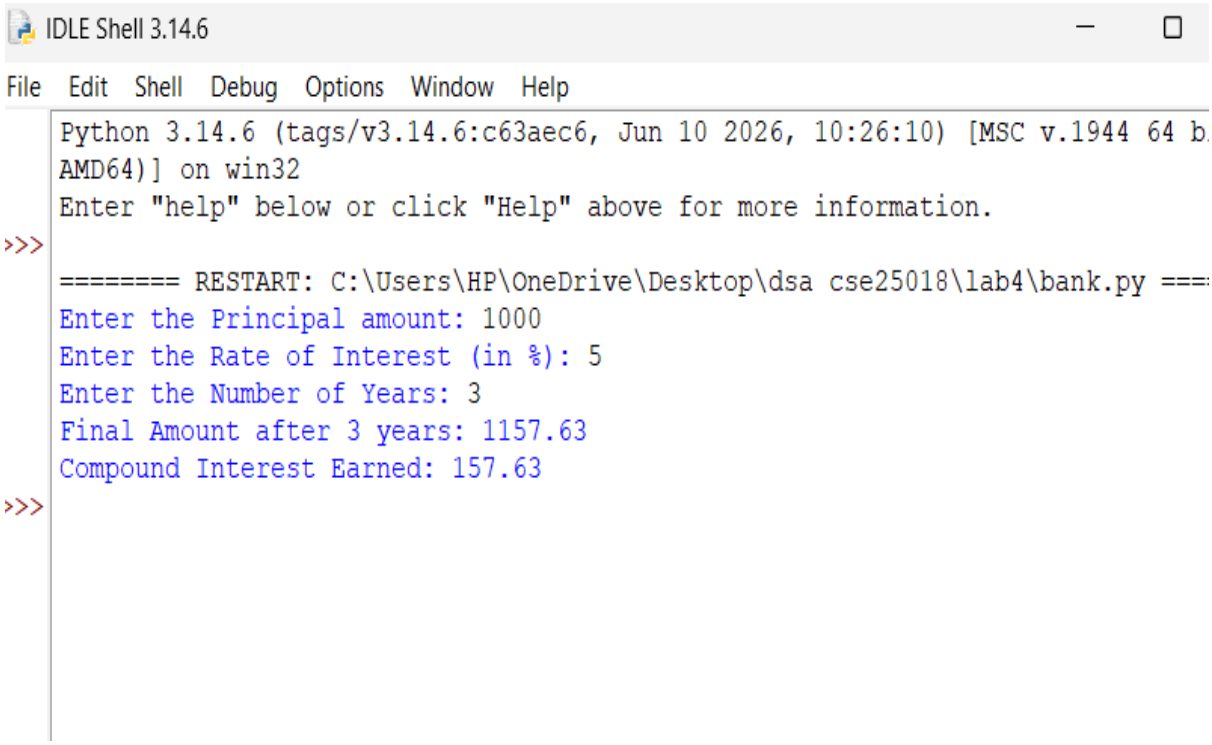
    # Using recursion to calculate (growth_factor)^years
    final_amount = principal * power(growth_factor, years)

    compound_interest_earned = final_amount - principal

    print(f"Final Amount after {years} years: {final_amount:.2f}")
    print(f"Compound Interest Earned: {compound_interest_earned:.2f}")


# Run the program
compound_interest()
```

Output:-



Explanation:-

The power function takes two parameters: P, the base, which stays the same across all calls, and n, the exponent, which decreases with each call. The base case checks whether n has become 0, and if so returns 1 directly, since any value raised to the power 0 is 1. Otherwise, the function returns P multiplied by the result of calling itself with the exponent reduced by one. Because this recursive call is part of a return statement, the current call cannot finish until the call it made returns a value, so each level of the recursion is left holding a pending multiplication by P until the base case is reached. Once the base case returns 1, that value is passed back to the call above it, which multiplies it by P and passes the result further up, and this continues until the outermost call returns. The final result is therefore built up only as the calls return, from the base case back to the original call, giving P multiplied by itself n times.

Error Table:-

S.No	*Error / Exception*	*Cause and Correction*
1	*No error*	----------

Experiment - 1(c)

Recursive Search for Employee ID in a List

Aim:-

An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Code:-

hr dept.py - C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\hr dept.py (3.14.6)

File Edit Format Run Options Window Help

```
def search_id(ids, target, i=0):
    if i == len(ids):           # reached end, not found
        return -1
    if ids[i] == target:        # found it
        return i
    return search_id(ids, target, i+1)    # check next one


# Employee ID list
employee_ids = [101, 102, 103, 104, 105]

target = int(input("Enter Employee ID to search: "))
position = search_id(employee_ids, target)

if position != -1:
    print("Found at position:", position + 1)
else:
    print("Not Found")
```

Output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>

====== RESTART: C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\hr dept.py ======
Enter Employee ID to search: 123
Not Found
>>
```

Explanation:-

The search function takes the list, the target employee ID to search for, and an index that tracks the current position, usually starting at 0. There are two separate base cases. The first checks whether the index has reached the length of the list, which means every element has already been checked without a match, and the function returns False. The second checks whether the element at the current index equals the target ID, and if so returns True immediately. When neither condition is satisfied, the function calls itself again with the index increased by one, and returns whatever that call produces. Because the result of the deeper call is returned directly without being modified, the answer found at any depth of the recursion is passed straight back up through every call above it unchanged. The recursion therefore moves forward through the list one position at a time, stopping as soon as a match is found or the list is exhausted.

Error Table:-

S.No	*Error / Exception*	*Cause and Correction*
1	*Indentation error*	*Corrected it by checking the gaps*
2	*----------*	*----------*
3	*----------*	*----------*

Experiment -1(d)-

Recursive Factorial Calculation

Aim:-

In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial number formula (n!). Develop a recursive program to calculate the factorial of a given number.

Code:-

log company lab-4.py - C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\log company la...

File Edit Format Run Options Window Help

```
# factorial of a given number
def fact(n):
    if n < 0:
        print("Factorial is not defined for negative numbers.")
        return
    elif n == 0 or n == 1:
        return 1
    else:
        return n * fact(n - 1)

n = int(input("Enter a number: "))

result = fact(n)

print(fact(n))
```

Output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\log company lab-4.py =
Enter a number: 5
120
>>>
```

Explanation:-

The factorial function checks whether n is 0 or 1 as its base case, since factorial is defined to stop at 1, and returns 1 directly in either case without further calls. For any other value, the function returns n multiplied by the result of calling itself with the argument reduced by one. As this call is inside a return statement, each call remains active on the stack, waiting for the call it made to finish, until the base case is reached. Once the base case returns 1, that value is passed back up, and each waiting call multiplies it by its own value of n before passing the result further up the chain. This means the multiplications only happen while the calls are returning, working from the smallest value back up to the original input, and the final value produced at the top of the stack is the factorial of the original number.

Error Table:-

S.No	*Error / Exception*	*Cause and Correction*
1	*No error*	*----------*
2	*No error*	*----------*

Experiment -1(e)

Recursive Fibonacci Series Generation

Aim:-

Write a recursive Python program to print the first n terms of the Fibonacci series.

Code:-

lab-4 fibb.py - C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\lab-4 fibb.py (3.14.6)

File Edit Format Run Options Window Help

```
n = int(input("please enter the number of terms of fibonacci series required to
# fibonacci series - 0,1,1,2,3,5,8,13,...........
# should only use the recursions instead of loops we need to write the functions
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(n):
    print(fibonacci(i), end=" ")

fibonacci(n)
```

Output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:\Users\HP\OneDrive\Desktop\dsa cse25018\lab4\lab-4 fibb.py ====
please enter the number of terms of fibonacci series required to be printed12
0 1 1 2 3 5 8 13 21 34 55 89
>>>
```

Explanation:-

The fibonacci function has two base cases: when n is 0 it returns 0, and when n is 1 it returns 1, since these are the two starting values of the series. For any other value of n, the function returns the sum of two recursive calls, one with n - 1 and one with n - 2, instead of making a single call as in the earlier examples. Each of these two calls again splits into two further calls unless it hits a base case, so the calls form a branching tree rather than a single chain. Every branch eventually reaches n == 0 or n == 1, returns the corresponding fixed value, and these returned values are added together and passed back up to whichever call made the two sub-calls. To print the first n terms, the function is called separately once for each position in the series, from 0 up to n - 1, and each returned value is printed in order, producing the familiar Fibonacci sequence.

Error Table:-

S.No	*Error / Exception*	*Cause and Correction*
1	----------	----------
2	----------	----------
3	----------	----------

Lab-2

Question 1: Linear Search

Question

Write a Python program to perform linear search on a list of elements entered by the user.

Aim:-

To implement and understand the working of the Linear Search algorithm, which searches for a target element by checking each element of the list sequentially from the beginning until the target is found or the list ends.

Code:-

*linear search.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-2\linear search.py (3....

File Edit Format Run Options Window Help

```
# write a python program to perform linear search on the given input

# taking input
n = int(input("Please enter no of elements in the array : "))
arr = []

for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

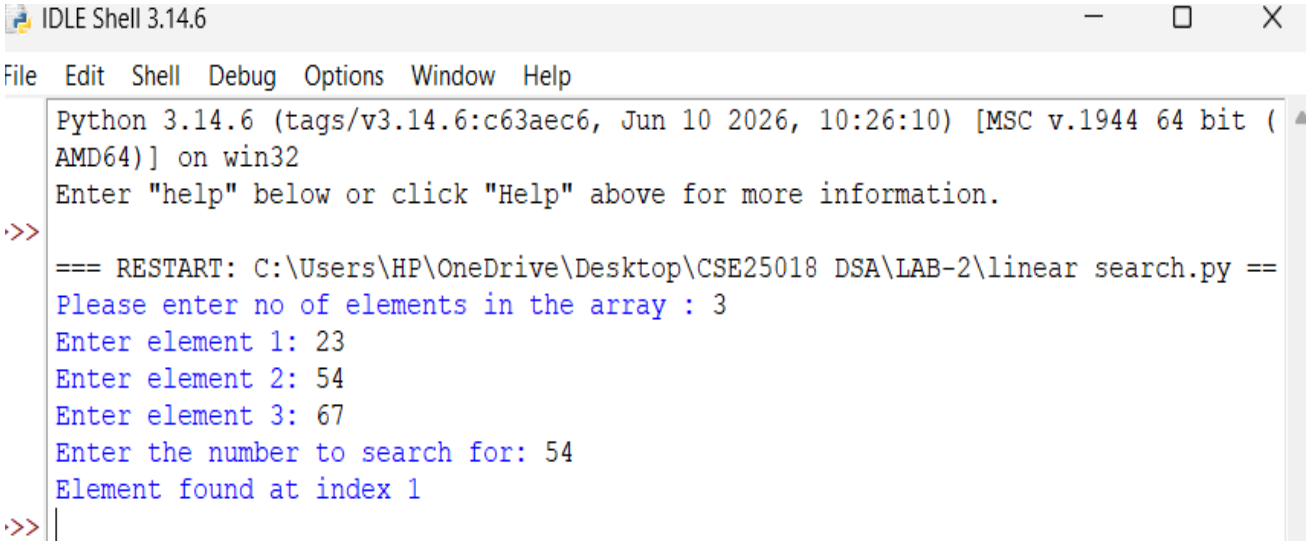

# linear search function
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1


target = int(input("Enter the number to search for: "))
result = linear_search(arr, target)

if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found in the list")
```

Document was last saved: Just now

Output:-



Explanation:-

The program first takes the number of elements and the elements themselves as input from the user and stores them in a list. The linear_search function then iterates through the list one element at a time, starting from index 0, and compares each element with the target value. If a match is found, the function immediately returns the index of that element. If the loop completes without finding a match, the function returns -1, indicating that the element is not present in the list. Linear search does not require the list to be sorted and works correctly on any arrangement of elements. Its time complexity is O(n) in the worst case, since every element may need to be checked once.

Error Table:-

S.No	*Error / Exception*	*Cause*	*Solution*
1	*ValueError*	*User enters a non-integer value when prompted for the number of elements or the array elements.*	*Enter only valid integer values when prompted, or add input validation using try-except.*
2	*Element not found*	*The target value does not exist anywhere in the entered list.*	*This is expected behaviour; the function correctly returns -1 and the program prints 'Element not found'.*
3	*Program appears to do nothing on large input*	*User enters a very large value of n, causing many repeated input prompts.*	*Enter a reasonable, intended value of n that matches the number of elements you plan to type.*

Question 2: Binary Search

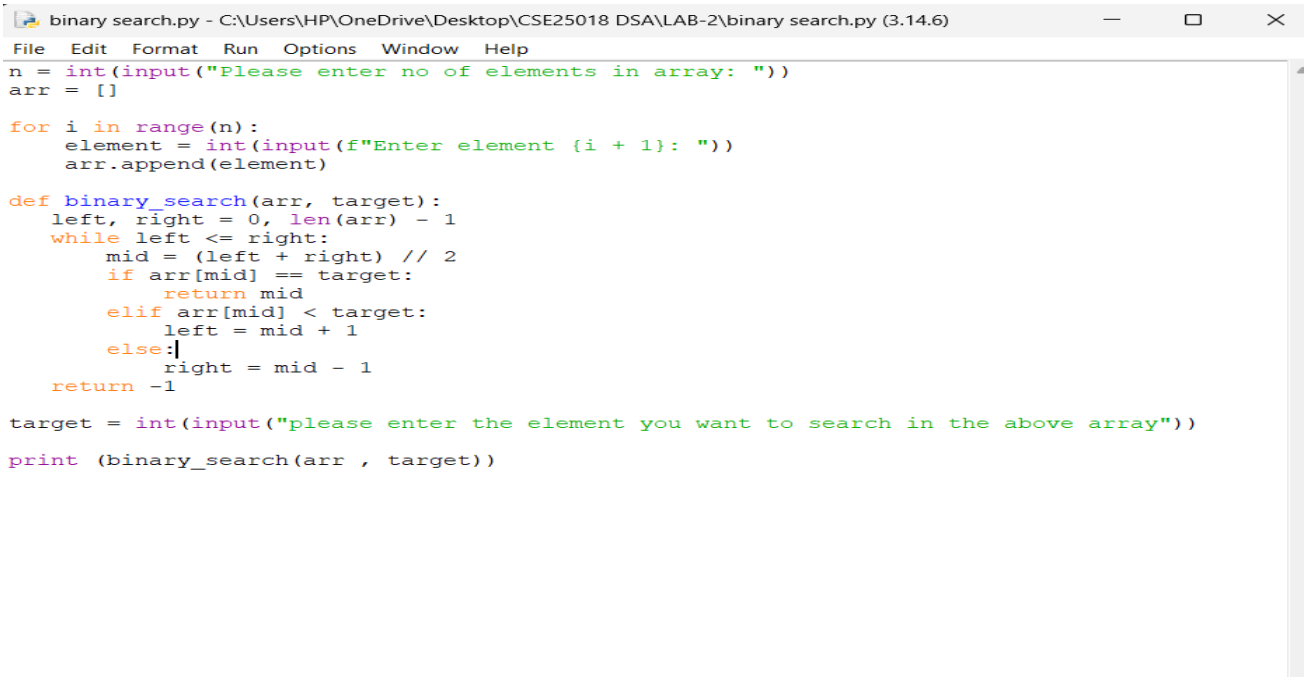
Question-

Write a Python program to perform binary search on a sorted list of elements entered by the user.

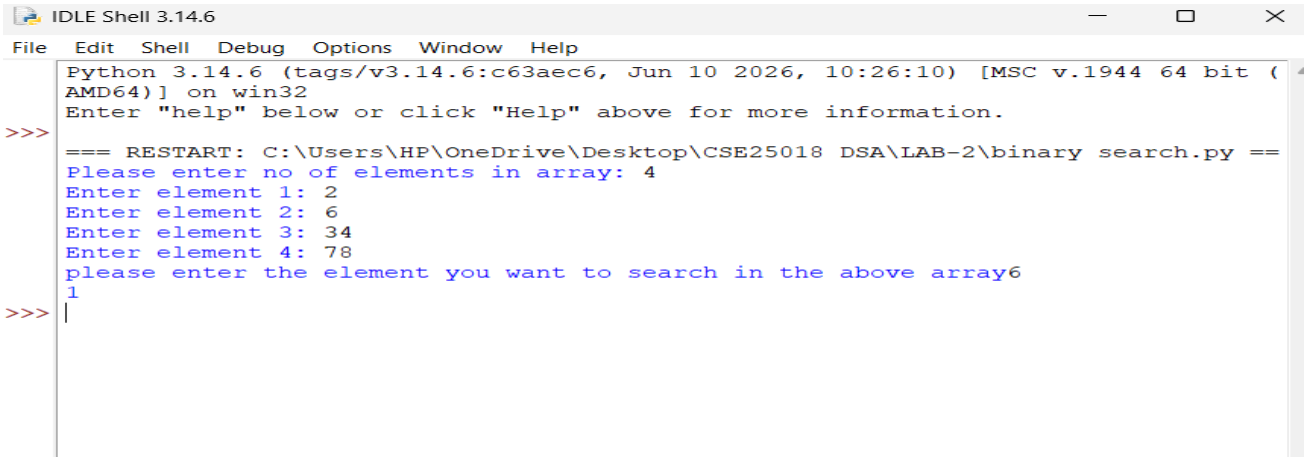
Aim:-

To implement and understand the working of the Binary Search algorithm, which efficiently locates a target element in a sorted list by repeatedly dividing the search interval in half.

Code:-



Output:-



Explanation:-

The program takes the number of elements and the array elements as input from the user. The binary_search function uses two pointers, left and right, that mark the current search boundaries within the list. In each iteration, it calculates the middle index, mid, and compares the element at that position with the target value. If they match, the index is returned. If the middle element is smaller than the target, the search continues in the right half by updating left to mid + 1; if it is larger, the search continues in the left half by updating right to mid - 1. This process repeats until the element is found or left exceeds right, in which case -1 is returned. Binary search requires the input list to be sorted in advance, and it is significantly faster than linear search for large datasets, running in O(log n) time compared to linear search's O(n).

Error Table:-

S.No	Error / Exception	Cause	Solution
1	Incorrect or missing result	The array entered by the user is not sorted, but binary search assumes a sorted array.	Sort the array (e.g., using arr.sort()) before calling binary_search, or ensure the user enters elements in ascending order.
2	ValueError	A non-integer value is entered for the number of elements, array elements, or the target.	Enter only valid integer values when prompted, or add input validation using try-except.
3	Result of search not displayed	The returned value of binary_search(arr, target) is not stored or printed.	Store the function's return value in a variable and print it, e.g., result = binary_search(arr, target); print(result).
4	Infinite loop	The left and right pointers are updated incorrectly, so the while condition never becomes false.	Ensure left is set to mid + 1 and right is set to mid - 1 exactly as required by the algorithm.

LAB-3

Question 1: Bubble Sort

Question-

Write a Python program to sort a list of elements entered by the user using the Bubble Sort algorithm.

Aim:-

To implement and understand the working of the Bubble Sort algorithm, which repeatedly compares adjacent elements and swaps them if they are in the wrong order, causing larger elements to 'bubble up' to the end of the list with each pass.

Code:-

BUBBLE SORT.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-3\BUBBLE SORT.py (...

File Edit Format Run Options Window Help

```
# Bubble Sort in Python (with user input)

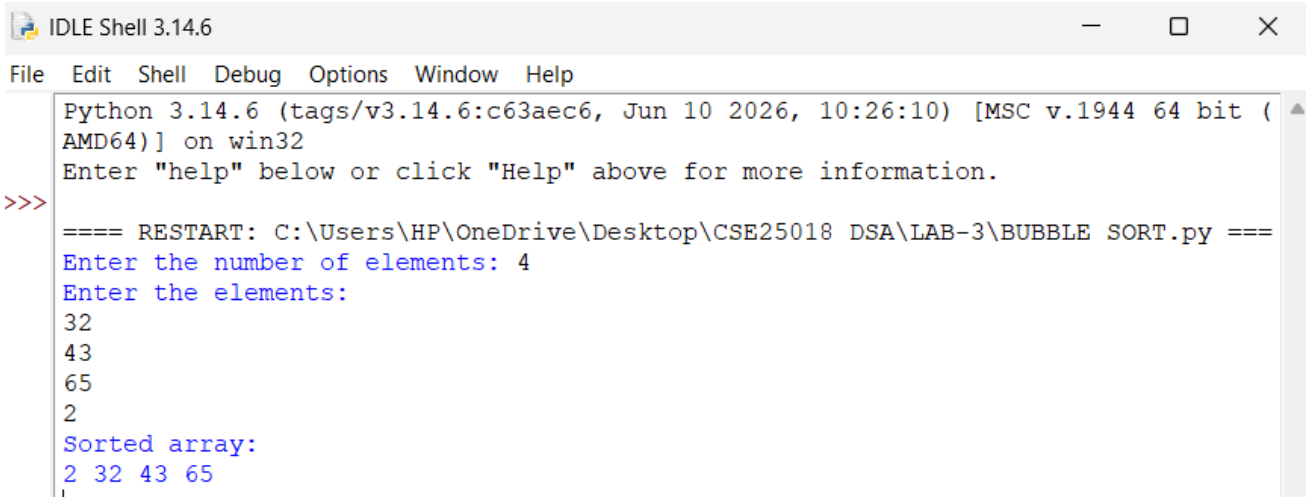
# Taking input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Bubble Sort
for i in range(n - 1):
    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

# Displaying the sorted array
print("Sorted array:")
for i in arr:
    print(i, end=" ")
```

Output:-



Explanation:-

The program first takes the number of elements and the elements themselves as input from the user and stores them in a list. The sorting logic uses two nested loops: the outer loop runs n - 1 times, representing each pass through the list, and the inner loop compares each pair of adjacent elements, arr[j] and arr[j + 1]. If an element is greater than the one next to it, the two are swapped. After each complete pass of the inner loop, the largest unsorted element is guaranteed to move to its correct position at the end of the array, so the range of the inner loop can safely shrink on each subsequent pass. This process continues until no more swaps are needed and the entire list is sorted in ascending order. Bubble sort has a time complexity of O(n^2) in the average and worst cases, making it simple to understand but inefficient for large datasets.

Error Table:-

S.No	*Error / Exception*	*Cause*	*Solution*
1	*ValueError*	*User enters a non-integer value when prompted for the number of elements or the array elements.*	*Enter only valid integer values when prompted, or add input validation using try-except.*
2	*Array remains unsorted / partially sorted*	*The inner loop range is written incorrectly (e.g., missing the '- i' term), so later passes re-compare already-sorted elements or skip needed comparisons.*	*Ensure the inner loop runs as range(n - i - 1) so each pass correctly narrows down to the unsorted portion of the list.*
3	*IndexError: list index out of range*	*The inner loop condition allows j + 1 to go beyond the last valid index of the array.*	*Keep the inner loop bound at n - i - 1 (not n) so that arr[j + 1] always refers to a valid index.*

Question 2: Insertion Sort

Question-

Write a Python program to sort a list of elements entered by the user using the Insertion Sort algorithm.

Aim:-

To implement and understand the working of the Insertion Sort algorithm, which builds a sorted list one element at a time by picking each element and inserting it into its correct position among the previously sorted elements.

Code:-

lab-3 insertion sort.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-3\lab-3 insertio...

File Edit Format Run Options Window Help

```
# Insertion Sort

n = int(input("Please enter the number of elements in the array: "))
arr = []

for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

def insertion_sort(arr):
    a = len(arr)

    for i in range(1, a):
        key = arr[i]
        j = i - 1

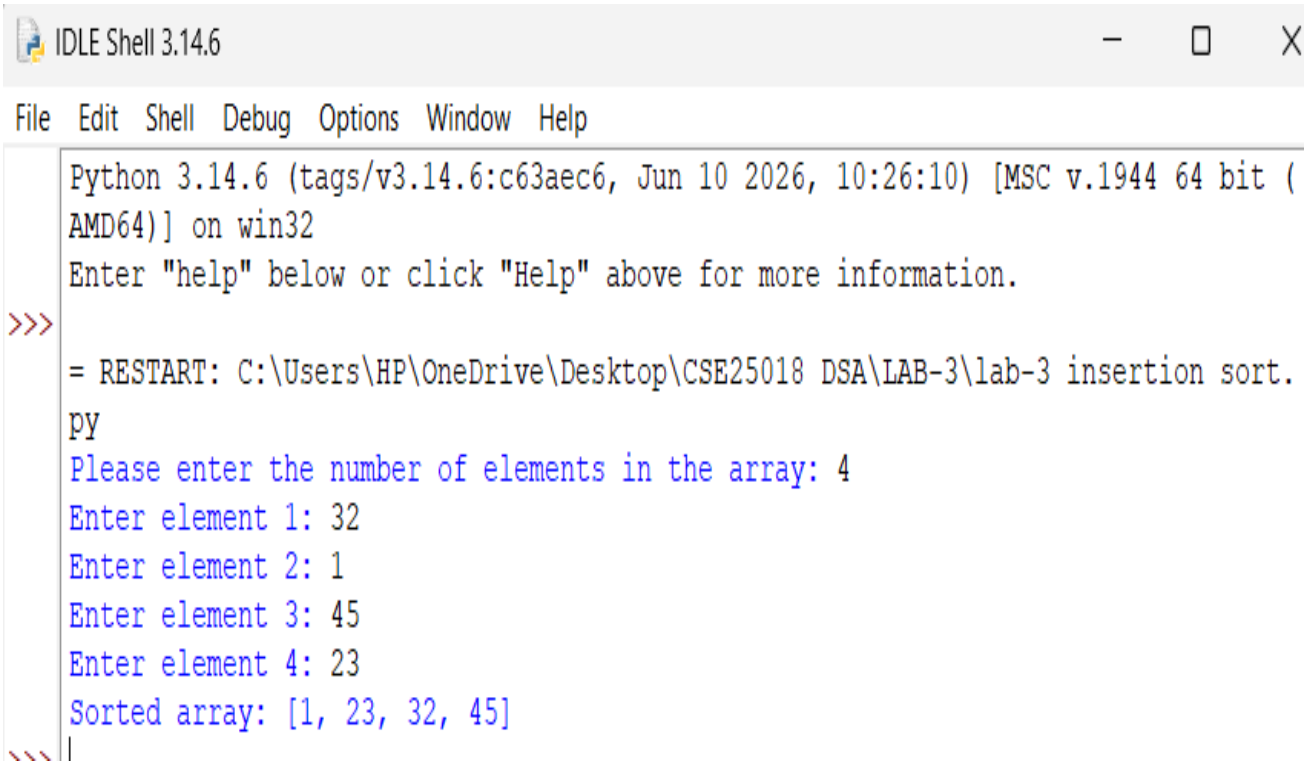
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr

insertion_sort(arr)
print("Sorted array:", arr)
```

Output:-



Explanation:-

The program takes the number of elements and the array elements as input from the user. The insertion_sort function starts from the second element (index 1) and treats it as the 'key' to be inserted into the sorted portion of the array that lies to its left. Using a while loop, it shifts each element in the sorted portion that is greater than the key one position to the right, creating space for the key. Once the correct position is found — that is, once j becomes less than 0 or arr[j] is no longer greater than the key — the key is placed at index j + 1. This process repeats for every element from index 1 to the end of the array, gradually growing the sorted portion until the entire list is sorted in ascending order. Insertion sort performs well on small or nearly-sorted lists, with a time complexity of O(n^2) in the worst case but close to O(n) when the input is already almost sorted.

Error Table:-

S.No	Error / Exception	Cause	Solution
1	ValueError	User enters a non-integer value when prompted for the number of elements or the array elements.	Enter only valid integer values when prompted, or add input validation using try-except.
2	Array remains unsorted	The while loop condition or the shifting step (arr[j + 1] = arr[j]) is written incorrectly, so elements are not shifted properly.	Ensure the while loop runs while j >= 0 and arr[j] > key, and that arr[j + 1] = key is placed correctly after the loop ends.
3	IndexError: list index out of range	The while loop condition checks arr[j] before confirming that j >= 0, causing access to a negative or invalid index.	Always check j >= 0 first in the while condition (Python evaluates conditions left to right, so order matters for short-circuiting).

Question 3: Merge Sort

Question-

Write a Python program to sort a list of elements entered by the user using the Merge Sort algorithm.

Aim:-

To implement and understand the working of the Merge Sort algorithm, which uses a divide-and-conquer approach to recursively split the list into halves, sort each half, and then merge the sorted halves back together.

Code:-

merge sort.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-3\merge sort.py (3.14.6)

File Edit Format Run Options Window Help

```
n = int(input("Please enter the number of elements in the array: "))
arr = []

for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)


def merge_sort(arr):
    if len(arr) <= 1:
        return

    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    # Recursively sort both halves
    merge_sort(left)
    merge_sort(right)

    i = j = k = 0

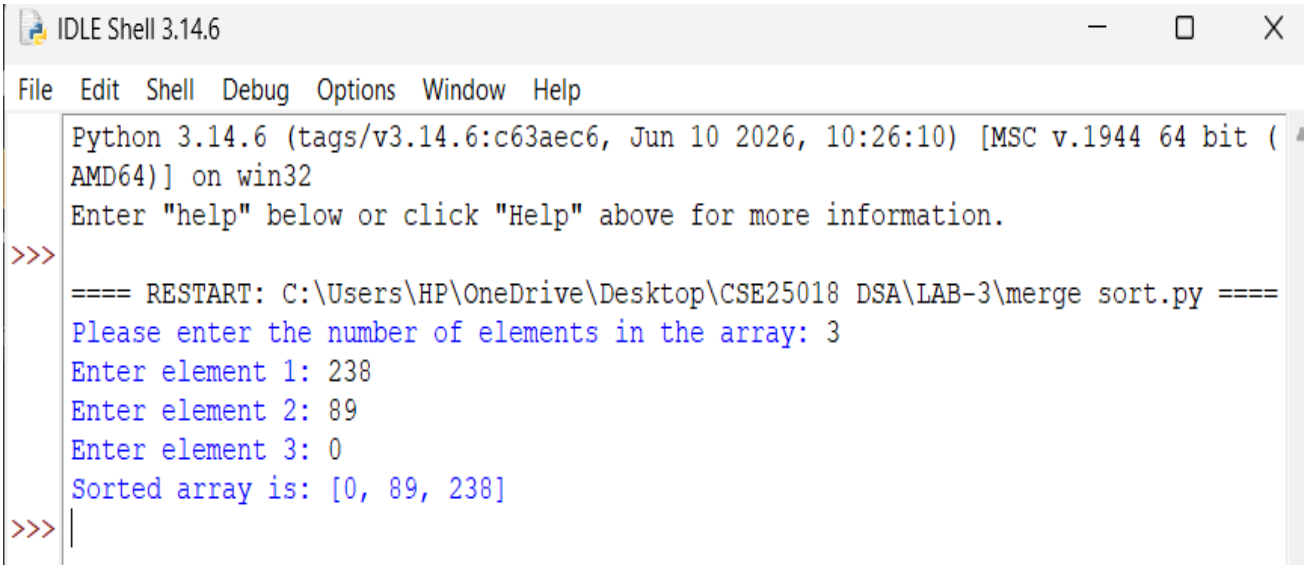
    # Merge the two sorted halves
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    # Copy remaining elements of left half
    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    # Copy remaining elements of right half
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
merge_sort(arr)

print("Sorted array is:", arr)
```

Output:-



Explanation:-

The program takes the number of elements and the array elements as input from the user. The merge_sort function first checks whether the list has one or zero elements, in which case it is already sorted and the function returns immediately. Otherwise, the list is divided into two halves, left and right, at the midpoint. The function then calls itself recursively on each half, breaking the problem down until every sub-list contains a single element. The merging step then combines two sorted halves into one sorted list by comparing the smallest remaining elements of each half and placing the smaller one into the original array at position k, advancing the corresponding pointer (i or j) each time. Once one half is fully placed, any remaining elements from the other half are copied over directly, since they are already sorted. This divide-and-conquer approach gives merge sort a time complexity of O(n log n) in all cases, making it considerably more efficient than bubble, selection, or insertion sort for large datasets, though it requires additional memory for the temporary left and right sub-lists.

Error Table:-

S.No	*Error / Exception*	*Cause*	*Solution*
1	*ValueError*	*User enters a non-integer value when prompted for the number of elements or the array elements.*	*Enter only valid integer values when prompted, or add input validation using try-except.*
2	*RecursionError: maximum recursion depth exceeded*	*The base case (if len(arr) <= 1: return) is missing or incorrect, so the recursive calls never stop.*	*Ensure the base case is present and correctly checks for lists of length 0 or 1 before dividing further.*
3	*Array remains unsorted or partially merged*	*The merging step's while loops or index variables (i, j, k) are updated incorrectly, so elements are skipped or overwritten.*	*Carefully update i, j, and k exactly once per comparison, and ensure all three 'while' loops (main merge and the two leftover-copy loops) are included.*

Question 4: Selection Sort

Question-

Write a Python program to sort a list of elements entered by the user using the Selection Sort algorithm.

Aim:-

To implement and understand the working of the Selection Sort algorithm, which repeatedly selects the smallest remaining element from the unsorted portion of the list and places it at the beginning of that portion.

Code:-

selection sort.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-3\selection sort.py (3...

File Edit Format Run Options Window Help

```
# selection sort

n = int(input("Please enter the number of elements in the array: "))
arr = []

for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)

def selection_sort(arr):
    a = len (arr)
    for i in range (a-1):
        min_index= i
        for j in range (i +1, a):
            if arr[j]<arr[min_index]:
                min_index  = j
                arr[i],arr[min_index] = arr[min_index],arr[i]
    return arr

selection_sort(arr)
print("sorted array is ", arr)
```

Output:

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
== RESTART: C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\LAB-3\selection sort.py ==
Please enter the number of elements in the array: 4
Enter element 1: 11
Enter element 2: 3
Enter element 3: 21
Enter element 4: 18
sorted array is  [3, 11, 18, 21]
```

Explanation:-

The program takes the number of elements and the array elements as input from the user. The selection_sort function uses an outer loop that runs from the first index to the second-last index, treating each position i as the start of the current unsorted portion. For each i, an inner loop scans through the remaining elements to find the index of the smallest value, keeping track of it in min_index. Once the smallest element in the unsorted portion has been located, it is swapped with the element at position i, placing it in its correct sorted position. This process repeats for each position, progressively growing the sorted portion at the front of the list until the entire array is sorted in ascending order. Selection sort has a time complexity of O(n^2) in all cases, since it always scans the remaining unsorted elements regardless of how nearly sorted the list already is, but it performs a minimal number of swaps compared to other quadratic sorting algorithms.

Error Table:-

S.No	*Error / Exception*	*Cause*	*Solution*
1	*ValueError*	*User enters a non-integer value when prompted for the number of elements or the array elements.*	*Enter only valid integer values when prompted, or add input validation using try-except.*

S.No	*Error / Exception*	*Cause*	*Solution*
2	*Array remains unsorted or incorrectly sorted*	*The swap statement is placed inside the inner loop instead of after it, so elements are swapped multiple times per pass instead of once with the final minimum.*	*Move the swap (arr[i], arr[min_index] = arr[min_index], arr[i]) outside and after the inner for loop, so it executes only once per outer loop iteration.*
3	*IndexError: list index out of range*	*The inner loop range is written incorrectly, causing j or min_index to exceed the valid index range of the array.*	*Ensure the inner loop runs as range(i + 1, a) so all indices accessed remain within bounds.*

Question 5: Quick Sort

Question-

Write a Python program to sort a list of elements entered by the user using the quick Sort algorithm.

Aim:-

To implement and understand the working of the quick sort algorithm and understand how the algorithm divides the array and sort step by step using iterations and loops

Code :-

QUICK SORT.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/QUICK SORT.py

File Edit Format Run Options Window Help

```
def quick_sort(arr):

    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = []
    right = []

    for x in arr[1:]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quick_sort(left) + [pivot] + quick_sort(right)


n = int(input("Enter number of elements: "))

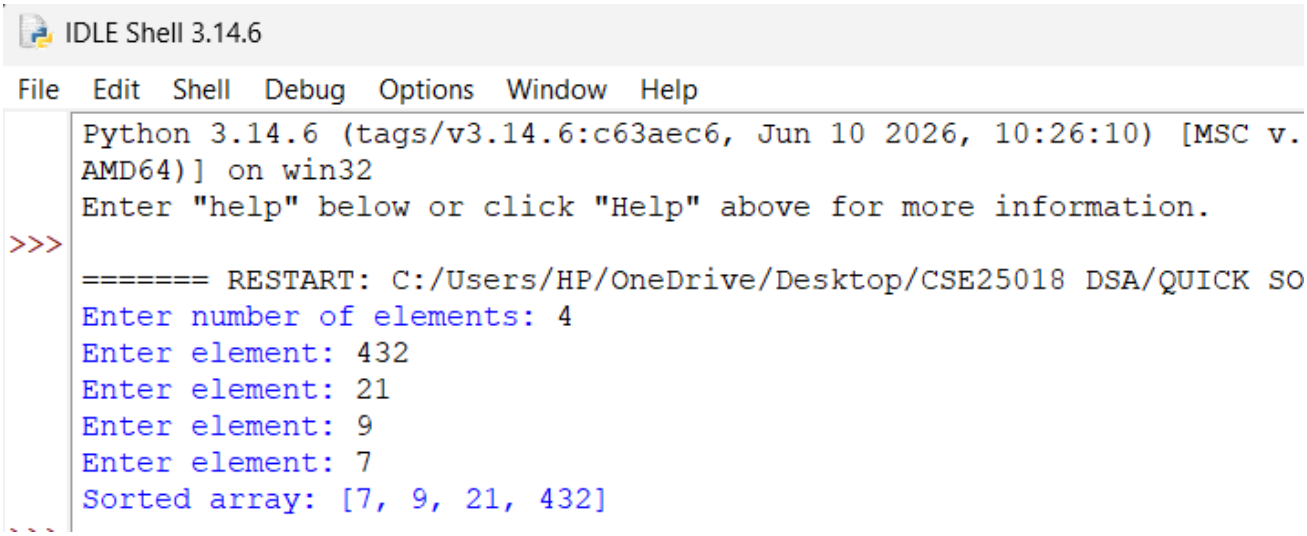
arr = []

for i in range(n):
    x = int(input("Enter element: "))
    arr.append(x)

arr = quick_sort(arr)

print("Sorted array:", arr)
```

Output:-



Explanation:-

Quick Sort is a divide-and-conquer sorting algorithm that selects an element as a pivot and divides the array into elements smaller than the pivot and elements greater than the pivot. It then recursively sorts these two parts and combines them with the pivot to produce the sorted array. Its average time complexity is O(n log n), while its worst-case complexity is O(n²).

Lab-4

Question 1:-

Write a Python program to implement a Singly Linked List using a menu-driven approach. The program should:

Create a singly linked list from an array of elements.

Insert a node at the beginning.

Insert a node at the end.

Insert a node at a specific index.

Delete a node from the beginning.

Delete a node from the end.

Delete a node containing a specific value.

Count the number of nodes.

Display the linked list.

Exit the program.

Aim:-

To implement a menu-driven Python program for a Singly Linked List and perform various operations such as insertion, deletion, counting, and displaying the elements of the linked list.

Code :-

*trip lab-4 insert in sll start original.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 insert in sll start origir

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SinglyLinkedList:

    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    # Insert at end
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    # Insert at specific index
    def insert_index(self, index, data):
        if index < 0 or index > self.count():
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        new = Node(data)
        temp = self.head

        for i in range(index - 1):
            temp = temp.next

        new.next = temp.next
        temp.next = new
```

*trip lab-4 insert in sll start original.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 insert in sll start orig

File Edit Format Run Options Window Help

```
        new.next = temp.next
        temp.next = new

    # Delete at beginning
    def delete_at_start(self):
        if self.head is None:
            print("No data to delete")
        else:
            print("Deleted value =", self.head.data)
            self.head = self.head.next

    # Delete at end
    def delete_at_end(self):
        if self.head is None:
            print("No data to delete")

        elif self.head.next is None:
            print("Deleted value =", self.head.data)
            self.head = None

        else:
            temp = self.head

            while temp.next.next:
                temp = temp.next

            print("Deleted value =", temp.next.data)
            temp.next = None

    # Delete specific value
    def delete_at_specific(self, value):
        if self.head is None:
            print("No data to delete")
            return

        if self.head.data == value:
            print("Deleted value =", self.head.data)
            self.head = self.head.next
            return

        temp = self.head

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
```

*trip lab-4 insert in sll start original.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 insert in sll start original.py (3.14.6

File Edit Format Run Options Window Help

```
while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Insert at beginning")
    print("2. Insert at end")
    print("3. Insert at specific index")
    print("4. Delete at beginning")
    print("5. Delete at end")
    print("6. Delete specific value")
    print("7. Count nodes")
    print("8. Display")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        sll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        sll.insert_end(data)

    elif choice == 3:
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))
        sll.insert_index(index, data)

    elif choice == 4:
        sll.delete_at_start()

    elif choice == 5:
        sll.delete_at_end()

    elif choice == 6:
        value = int(input("Enter value to delete: "))
        sll.delete_at_specific(value)

    elif choice == 7:
        print("Number of nodes =", sll.count())

    elif choice == 8:
        sll.display()

    elif choice == 9:
        print("Program ended")
```

*trip lab-4 insert in sll start original.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 insert in s

File Edit Format Run Options Window Help

```
        if temp.next is None:
            print("Value not present")
        else:
            print("Deleted value =", temp.next.data)
            temp.next = temp.next.next

    # Count nodes
    def count(self):
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        return c

    # Display
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return

        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")

sll = SinglyLinkedList()
n = int(input("Enter number of elements: "))

arr = list(map(int, input("Enter elements: ").split()))

# Check size
if len(arr) != n:
    print("Number of elements entered is not equal to n")
else:

    # Convert array into linked list
    for value in arr:
        sll.insert_end(value)

    print("\nInitial Linked List:")
    sll.display()
```

*trip lab-4 insert in sll start original.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 inser

File Edit Format Run Options Window Help

```
while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Insert at beginning")
    print("2. Insert at end")
    print("3. Insert at specific index")
    print("4. Delete at beginning")
    print("5. Delete at end")
    print("6. Delete specific value")
    print("7. Count nodes")
    print("8. Display")
    print("9. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        sll.insert_begin(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        sll.insert_end(data)

    elif choice == 3:
        index = int(input("Enter index: "))
        data = int(input("Enter data: "))
        sll.insert_index(index, data)

    elif choice == 4:
        sll.delete_at_start()

    elif choice == 5:
        sll.delete_at_end()

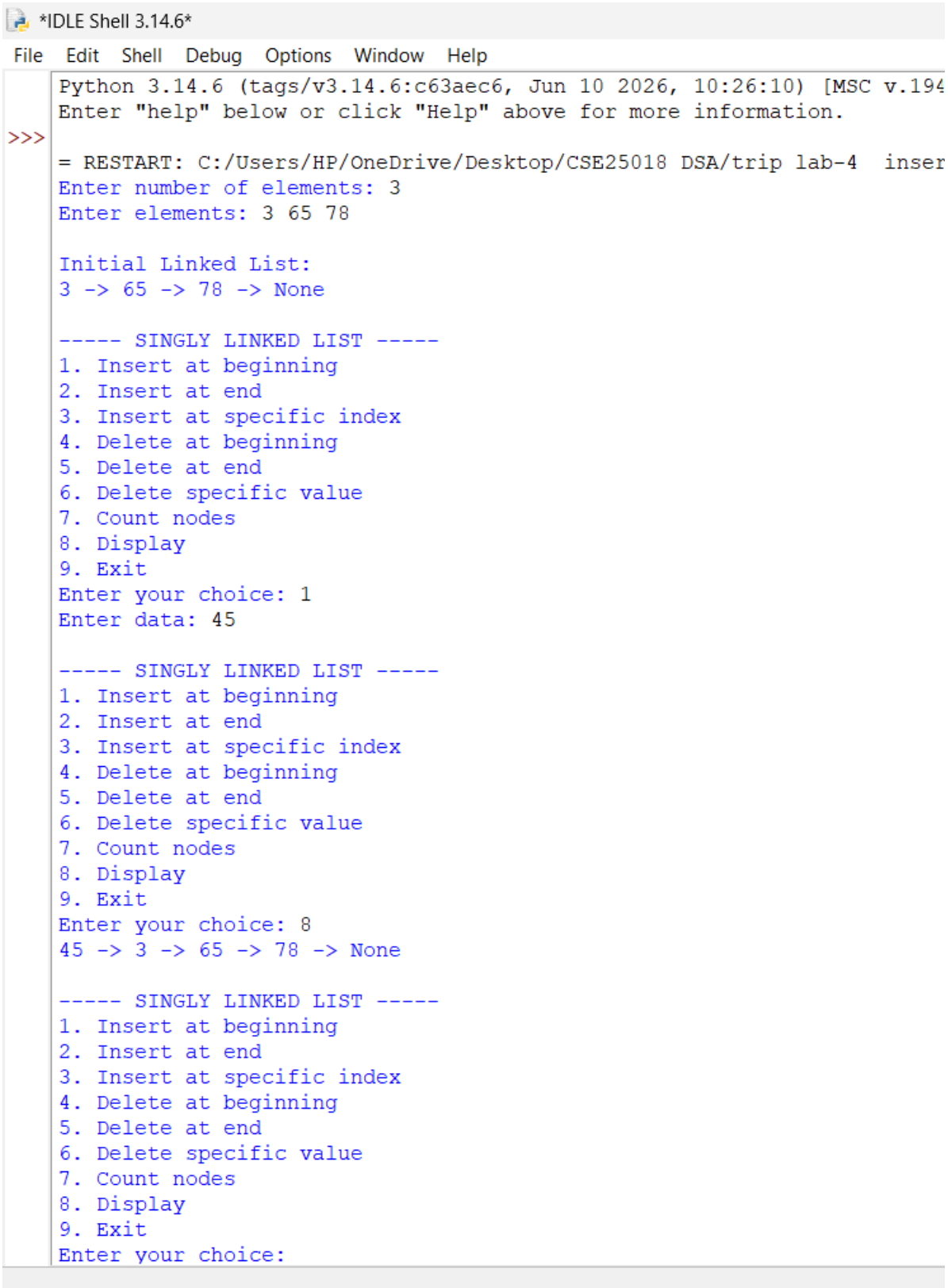
    elif choice == 6:
        value = int(input("Enter value to delete: "))
        sll.delete_at_specific(value)

    elif choice == 7:
        print("Number of nodes =", sll.count())

    elif choice == 8:
        sll.display()

    elif choice == 9:
        print("Program ended")
```

Code Output:-


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.194
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/trip lab-4 inser
Enter number of elements: 3
Enter elements: 3 65 78
Initial Linked List:
3 -> 65 -> 78 -> None
----- SINGLY LINKED LIST -----
1. Insert at beginning
2. Insert at end
3. Insert at specific index
4. Delete at beginning
5. Delete at end
6. Delete specific value
7. Count nodes
8. Display
9. Exit
Enter your choice: 1
Enter data: 45
----- SINGLY LINKED LIST -----
1. Insert at beginning
2. Insert at end
3. Insert at specific index
4. Delete at beginning
5. Delete at end
6. Delete specific value
7. Count nodes
8. Display
9. Exit
Enter your choice: 8
45 -> 3 -> 65 -> 78 -> None
----- SINGLY LINKED LIST -----
1. Insert at beginning
2. Insert at end
3. Insert at specific index
4. Delete at beginning
5. Delete at end
6. Delete specific value
7. Count nodes
8. Display
9. Exit
Enter your choice:

Explanation:-

This program implements a Singly Linked List in Python using a menu-driven approach. It first takes array elements as input and converts them into linked-list nodes. The program provides operations to insert nodes at the beginning, end, or a specific index, and to delete nodes from the beginning, end, or by a specific value. It can also count the number of nodes and display the linked list.

Error Table:-

ERROR	*Possible Cause*	*Solution*
List Underflow	*delete() or traverse() called when the list is empty (head is None)*	*Check that head is not None before deleting or traversing; insert a node first*
AttributeError: 'NoneType' object has no attribute 'next'	*Accessing .next on a node that is already None (e.g., traversing past the last node)*	*Check that the current node is not None before accessing current.next*
Node not found during deletion	*The value to be deleted does not exist anywhere in the list*	*Search the list first and print a "not found" message if the value is absent*

Question 4(b):

Write a Python program to implement a Doubly Linked List using a menu-driven approach. The program should:

1. *Create a singly linked list from an array of elements.*
2. *Insert a node at the beginning.*
3. *Insert a node at the end.*
4. *Insert a node at a specific index.*
5. *Delete a node from the beginning.*
6. *Delete a node from the end.*
7. *Delete a node containing a specific value.*
8. *Count the number of nodes.*
9. *Display the linked list.*
10. *Exit the program.*

Aim

To implement a menu-driven Python program for a Doubly Linked List and perform various operations such as insertion, deletion, counting, and displaying the elements of the linked list.

Code :

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node

        print("Node inserted at beginning.")

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            print("Node inserted at end.")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp

        print("Node inserted at end.")

    # Insert at a position
    def insert_position(self, data, position):
        if position <= 0:
            print("Invalid position.")
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
        print("Node inserted at end.")

    # Insert at a position
    def insert_position(self, data, position):
        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.insert_beginning(data)
            return

        new_node = Node(data)
        temp = self.head

        for i in range(1, position - 1):
            if temp is None:
                print("Position does not exist.")
                return
            temp = temp.next

        if temp is None:
            print("Position does not exist.")
            return

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted at position", position)

    # Delete from beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
    # Delete from a position
    def delete_position(self, position):
        if self.head is None:
            print("List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.delete_beginning()
            return

        temp = self.head

        for i in range(1, position):
            if temp is None:
                print("Position does not exist.")
                return
            temp = temp.next

        if temp is None:
            print("Position does not exist.")
            return

        temp.prev.next = temp.next

        if temp.next is not None:
            temp.next.prev = temp.prev

        print("Deleted:", temp.data)

    # Display forward
    def display_forward(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Forward:", end=" ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Forward:", end=" ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("NULL")

    # Display backward
    def display_backward(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        print("Backward:", end=" ")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.prev

        print("NULL")

    # Search
    def search(self, value):
        temp = self.head
        position = 1

        while temp is not None:
            if temp.data == value:
                print("Element found at position:", position)
                return

            temp = temp.next
            position += 1

        print("Element not found.")
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
        temp = self.head
        position = 1

        while temp is not None:
            if temp.data == value:
                print("Element found at position:", position)
                return

            temp = temp.next
            position += 1

        print("Element not found.")

    # Update
    def update(self, position, new_data):
        if self.head is None:
            print("List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        temp = self.head

        for i in range(1, position):
            if temp is None:
                print("Position does not exist.")
                return
            temp = temp.next

        if temp is None:
            print("Position does not exist.")
            return

        old_data = temp.data
        temp.data = new_data

        print("Updated", old_data, "to", new_data)

    # Count nodes
    def count(self):
        temp = self.head
        count = 0

        while temp is not None:
            count += 1
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
    # Count nodes
    def count(self):
        temp = self.head
        count = 0

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)


# MAIN PROGRAM

dll = DoublyLinkedList()

while True:

    print("\n========== DOUBLY LINKED LIST ==========")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Insert at Position")
    print("4. Delete from Beginning")
    print("5. Delete from End")
    print("6. Delete from Position")
    print("7. Display Forward")
    print("8. Display Backward")
    print("9. Search")
    print("10. Update")
    print("11. Count Nodes")
    print("12. Exit")
    print("========================================")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        dll.insert_beginning(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        dll.insert_position(data, position)
```

lab-5.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-5.py (3.14.6)

File Edit Format Run Options Window Help

```
        data = int(input("Enter data: "))
        dll.insert_beginning(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        dll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        dll.insert_position(data, position)

    elif choice == 4:
        dll.delete_beginning()

    elif choice == 5:
        dll.delete_end()

    elif choice == 6:
        position = int(input("Enter position to delete: "))
        dll.delete_position(position)

    elif choice == 7:
        dll.display_forward()

    elif choice == 8:
        dll.display_backward()

    elif choice == 9:
        value = int(input("Enter value to search: "))
        dll.search(value)

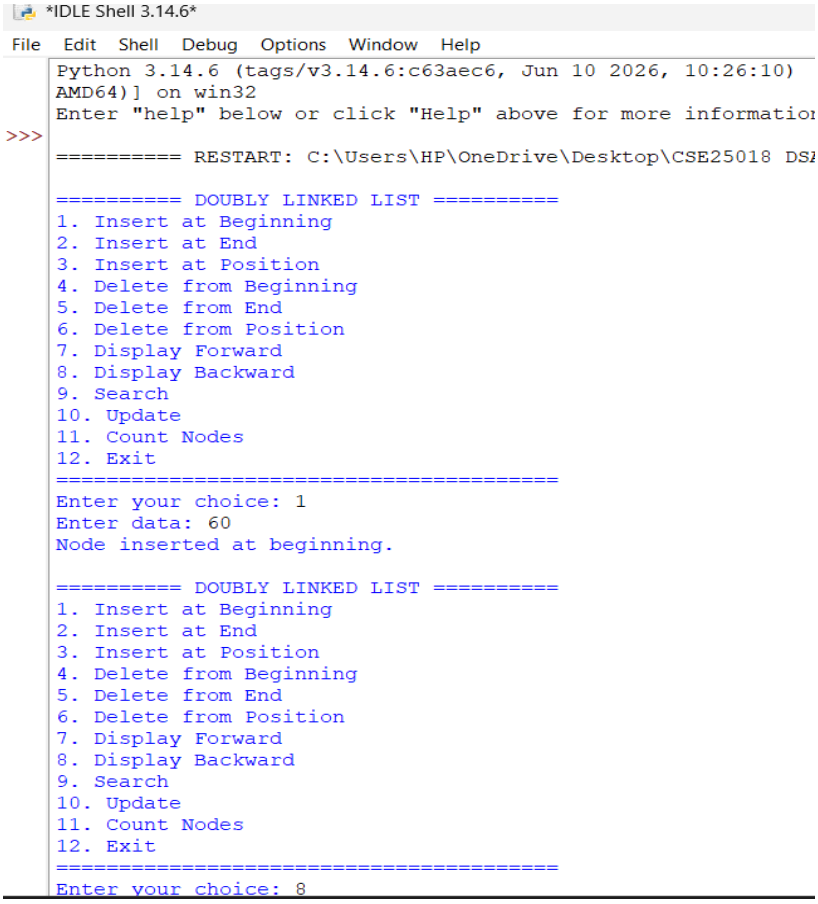
    elif choice == 10:
        position = int(input("Enter position to update: "))
        new_data = int(input("Enter new value: "))
        dll.update(position, new_data)

    elif choice == 11:
        dll.count()

    elif choice == 12:
        print("Program ended.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Code Output:-



Explanation:-

In Doubly Linked List code, a Node has data, prev and next pointers. head pointer holds the first node of list. next pointer of a node is used to point to next node in list. prev pointer of a node is used to point to previous node in list. During insertion and deletion both the pointers are updated to form links between nodes. A while loop is used to traverse through list of nodes. In DLL code, we can insert, delete, search and display elements in list. These operations are performed when we require to move in list in forward as well as backward direction.

Error Table:-

ERROR	*POSSIBLE SOLUTION*	*SOLUTION*
List Underflow	*delete() or traverse() called when the list is empty (head is None)*	*Check that head is not None before deleting or traversing*
Dangling tail pointer	*tail is not updated after the last node is deleted*	*Update tail to point to the previous node whenever the last node is removed*

Question-3-

Write a Python program to implement a Circularly Linked List using a menu-driven approach. The program should:

1. *Create a circular linked list from an array of elements.*
2. *Insert a node at the beginning.*
3. *Insert a node at the end.*
4. *Insert a node at a specific index.*
5. *Delete a node from the beginning.*
6. *Delete a node from the end.*
7. *Delete a node containing a specific value.*
8. *Count the number of nodes.*
9. *Display the linked list.*
10. *Exit the program.*

Aim:-

To implement a menu-driven Python program for a Circular Linked List and perform various operations such as insertion, deletion, counting, and displaying the elements of the linked list.

Code:-

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None

    # Insert at beginning
    def insert_beginning(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

        print("Node inserted at beginning.")

    # Insert at end
    def insert_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = new_node
            new_node.next = self.head

        print("Node inserted at end.")
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
        print("Node inserted at end.")

    # Insert at position
    def insert_position(self, data, position):
        if position <= 0:
            print("Invalid position.")
            return

        if self.head is None and position != 1:
            print("Position does not exist.")
            return

        if position == 1:
            self.insert_beginning(data)
            return

        new_node = Node(data)
        temp = self.head

        for i in range(1, position - 1):
            temp = temp.next

            if temp == self.head:
                print("Position does not exist.")
                return

        new_node.next = temp.next
        temp.next = new_node

        print("Node inserted at position", position)

    # Delete from beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted:", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted:", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        deleted = self.head.data

        temp.next = self.head.next
        self.head = self.head.next

        print("Deleted:", deleted)

    # Delete from end
    def delete_end(self):
        if self.head is None:
            print("List is empty.")
            return

        # Only one node
        if self.head.next == self.head:
            print("Deleted:", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next

        deleted = temp.next.data
        temp.next = self.head

        print("Deleted:", deleted)

    # Delete from position
    def delete_position(self, position):
        if self.head is None:
            print("List is empty.")
            return
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
        print("Deleted:", deleted)

    # Delete from position
    def delete_position(self, position):
        if self.head is None:
            print("List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.delete_beginning()
            return

        temp = self.head

        for i in range(1, position - 1):
            temp = temp.next

            if temp == self.head:
                print("Position does not exist.")
                return

        if temp.next == self.head:
            print("Position does not exist.")
            return

        deleted = temp.next.data
        temp.next = temp.next.next

        print("Deleted:", deleted)

    # Display
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Circular Linked List:", end=" ")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
    def display(self):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head

        print("Circular Linked List:", end=" ")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")

    # Search
    def search(self, value):
        if self.head is None:
            print("List is empty.")
            return

        temp = self.head
        position = 1

        while True:
            if temp.data == value:
                print("Element found at position:", position)
                return

            temp = temp.next
            position += 1

            if temp == self.head:
                break

        print("Element not found.")

    # Update
    def update(self, position, new_data):
        if self.head is None:
            print("List is empty.")
            return

        if position <= 0:
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
    # Update
    def update(self, position, new_data):
        if self.head is None:
            print("List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        temp = self.head
        current_position = 1

        while current_position < position:
            temp = temp.next

            if temp == self.head:
                print("Position does not exist.")
                return

            current_position += 1

        old_data = temp.data
        temp.data = new_data

        print("Updated", old_data, "to", new_data)

    # Count nodes
    def count(self):
        if self.head is None:
            print("Number of nodes: 0")
            return

        temp = self.head
        count = 0

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)


# MAIN PROGRAM
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
# MAIN PROGRAM

cll = CircularLinkedList()

while True:

    print("\n========== CIRCULAR LINKED LIST ==========")
    print("1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Insert at Position")
    print("4. Delete from Beginning")
    print("5. Delete from End")
    print("6. Delete from Position")
    print("7. Display")
    print("8. Search")
    print("9. Update")
    print("10. Count Nodes")
    print("11. Exit")
    print("==========================================")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        cll.insert_beginning(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        cll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        cll.insert_position(data, position)

    elif choice == 4:
        cll.delete_beginning()

    elif choice == 5:
        cll.delete_end()

    elif choice == 6:
        position = int(input("Enter position to delete: "))
        cll.delete_position(position)

    elif choice == 7:
        cll.display()
```

lab 5 cll.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py (3.14.6)

File Edit Format Run Options Window Help

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter data: "))
        cll.insert_beginning(data)

    elif choice == 2:
        data = int(input("Enter data: "))
        cll.insert_end(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        cll.insert_position(data, position)

    elif choice == 4:
        cll.delete_beginning()

    elif choice == 5:
        cll.delete_end()

    elif choice == 6:
        position = int(input("Enter position to delete: "))
        cll.delete_position(position)

    elif choice == 7:
        cll.display()

    elif choice == 8:
        value = int(input("Enter value to search: "))
        cll.search(value)

    elif choice == 9:
        position = int(input("Enter position to update: "))
        new_data = int(input("Enter new value: "))
        cll.update(position, new_data)

    elif choice == 10:
        cll.count()

    elif choice == 11:
        print("Program ended.")
        break

    else:
        print("Invalid choice. Please try again.")
```

Code output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab 5 cll.py ==

========== CIRCULAR LINKED LIST ==========
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display
8. Search
9. Update
10. Count Nodes
11. Exit
==========================================
Enter your choice: 1
Enter data: 32
Node inserted at beginning.

========== CIRCULAR LINKED LIST ==========
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display
8. Search
9. Update
10. Count Nodes
11. Exit
==========================================
Enter your choice: 7
Circular Linked List: 32 -> HEAD

========== CIRCULAR LINKED LIST ==========
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display
8. Search
```

Explanation:-

In Circular Linked List code, a Node class is used to create each node containing data and a next pointer. The head pointer stores the first node of the list. During insertion, the new node is connected to another node using the next pointer. The last node always points back to the head, making the list circular. A while loop is commonly used to traverse the list until we reach the head again. CLL code is useful for insertion, deletion, searching, and displaying elements. It is mainly used when data needs to be processed repeatedly in a circular manner.

Error Table:-

ERROR	*POSSIBLE SOLUTION*	*SOLUTION*
Infinite loop	*Loop condition checks while node is not None but in a circular list the last node points back to head, never to None*	*Use while node is not head checked after the first step or a do-while style loop*
Last node not linked back to head	*Forgetting to set the new last node's next to head after insertion*	*Always update tail.next = head immediately after every insertion or deletion*
Single-node deletion error	*Deleting the only node in the list without resetting head to None*	*Explicitly check if head.next == head (only one node) and set head = None in that case*

LAB- 5

1. Program to Implement Stack Using Linked List

Aim:-

To write a Python program to implement the operations of a Stack (Push, Pop, Peek, and Display) using a singly linked list, and to verify its working through a menu-driven interface.

Code:-

STACKS.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/STACKS.py (3.14.6)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:
    def __init__(self):
        self.top = None

    # Push operation
    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    # Pop operation
    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next


# Create stack object outside the class
s = Stack()
```

STACKS.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/STACKS.py (3.14.6)

File Edit Format Run Options Window Help

```
    # Peek operation
    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    # Display operation
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next


# Create stack object outside the class
s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Code Output:-



Explanation:-

The program implements a Stack using a linked list instead of a fixed-size array, so it can grow dynamically. The Node class defines a single unit of the list, storing a data

value and a reference (next) to the following node. The Stack class maintains only one pointer, top, which always refers to the most recently inserted node.The push() method creates a new node, links it to the current top, and then updates top to point to this new node — this places every new element at the front of the list, which corresponds to the top of the stack, giving O(1) insertion. The pop() method checks whether top is None (an empty stack, called Stack Underflow); if not empty, it prints the data of the current top node and then moves top to the next node, effectively removing the old top.The peek() method simply reports the data stored at top without removing it, while display() traverses the linked list from top to the last node, printing each element, thereby showing the entire stack content in Last-In-First-Out (LIFO) order. Finally, the menu-driven loop repeatedly asks the user to choose an operation and calls the corresponding method until the user selects the Exit option (choice 5), at which point the loop breaks and the program terminates.

Error Table:-

Error	*Possible Cause*	*Solution*
Stack Underflow	*pop() or peek() is called when the stack is empty (top is None)*	*Check if the stack is empty before performing pop/peek; push an element first*
ValueError: invalid literal for int()	*Non-numeric text entered at the choice or element prompt*	*Enter only numeric values, or wrap input() in a try–except block*
"Invalid choice" printed repeatedly	*A menu option outside the range 1–5 is entered*	*Enter a valid option between 1 and 5*
Program does not terminate	*Option 5 (Exit) is never selected*	*Select choice 5 to break out of the while loop*

2. program to implement stacks using array

Aim:-

To write a Python program to implement the operations of a Stack (Push, Pop, Peek, and Display) using an array , and to verify its working through a menu-driven interface.

Code:-

stack using array.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/stack using array.py (3...

File Edit Format Run Options Window Help

```
stack = []
top = -1

# Push
stack.append(10)
top = top + 1

stack.append(20)
top = top + 1

stack.append(30)
top = top + 1

print(stack)

# Pop
if top == -1:
    print("Stack Underflow")
else:
    print("Deleted:", stack.pop())
    top = top - 1

print(stack)

# Peek
if top == -1:
    print("Stack is Empty")
else:
    print("Top:", stack[top])
```

Code Output:-

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:
AMD64)] on win32
Enter "help" below or click "Help" above for more inform
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/
[10, 20, 30]
Deleted: 30
[10, 20]
Top: 20
>>>
```

Explanation:-

The stack using an array is implemented using a Python list, where the elements are stored in the order they are inserted. The variable top keeps track of the position of the

last element in the stack and initially it is set to -1, which indicates that the stack is empty. During the push operation, an element is added using append() and top is increased by one. During the pop operation, the top element is removed using pop() and top is decreased by one. The peek operation displays the element present at the top position without removing it. Since a stack follows LIFO (Last In, First Out), the element inserted last is always removed first

ERROR TABLE:

ERROR	*SOLUTION*	
Stack Using Array – Error	*Meaning*	*Condition*
Stack Overflow	*Trying to insert an element when the stack is full*	*top == MAX - 1*
Stack Underflow	*Trying to delete an element when the stack is empty*	*top == -1*
Invalid Peek	*Trying to view the top element when the stack is empty*	*top == -1*

Lab-6

1(a). Program to Implement Queue Using Linked List

Aim:-

To write a Python program to implement the operations of a Queue (Enqueue, Dequeue, Peek, and Display) using a singly linked list, and to verify its working through a menu-driven interface

Code:-

QUEUE.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/QUEUE.py (3.14.6)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class QueueEx:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue operation
    def enqueue(self, x):
        new = Node(x)
        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new
        print(x, "inserted into the queue")

    # Dequeue operation
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            x = self.front.data
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(f"{x} deleted from the queue")

    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while temp is not None:
```

QUEUE.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/QUEUE.py (3.14.6)

File Edit Format Run Options Window Help

```
    # Peek operation
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

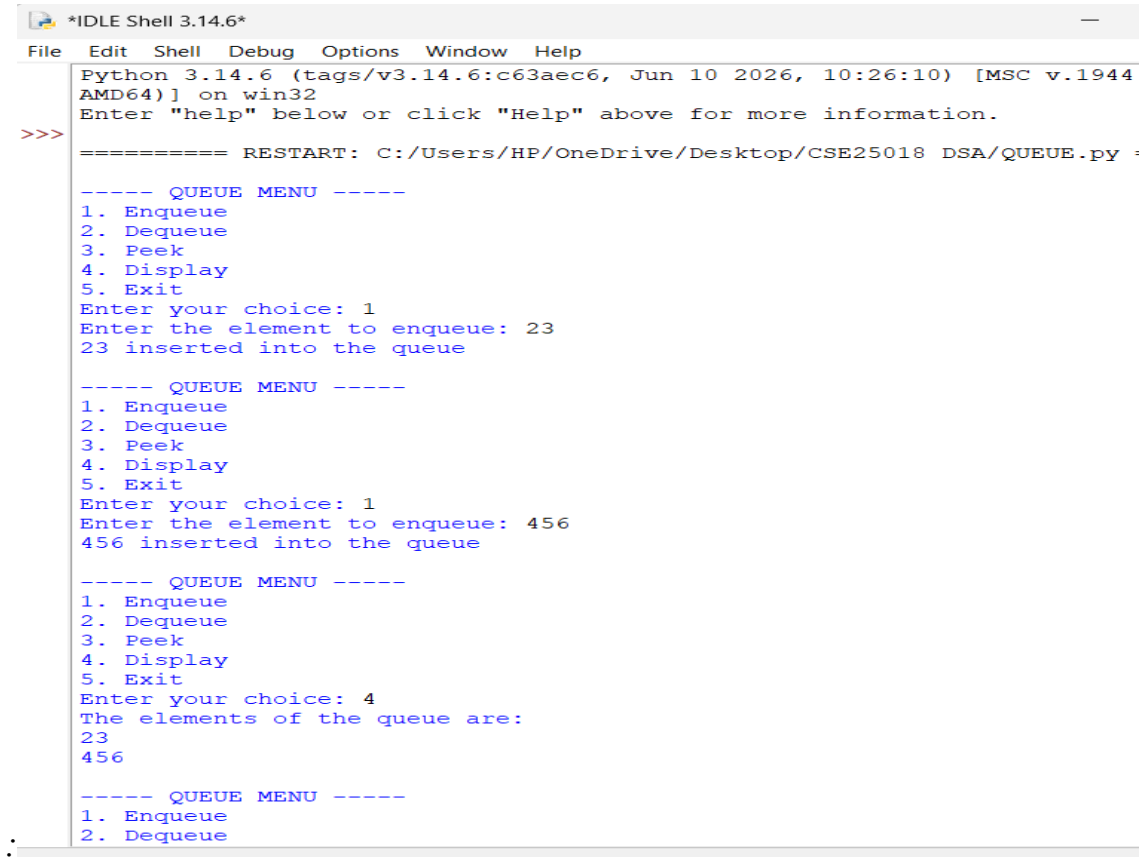
    # Display operation
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while temp is not None:
                print(temp.data)
                temp = temp.next


# Menu-driven program
q = QueueEx()

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

Code output:-



:

Explanation:-

This program implements a Queue using a linked list, allowing First-In-First-Out (FIFO) behaviour with dynamic size. The QueueEx class keeps two pointers: front, which refers to the node that will be removed next, and rear, which refers to the most recently added node — this lets both insertion and removal be performed in O(1) time without traversing the list.The enqueue() method creates a new node and, if the queue is currently empty (rear is None), sets both front and rear to this new node; otherwise, it links the new node after the current rear and then updates rear to point to it, so new elements always join at the back of the queue. The dequeue() method first checks whether front is None (an empty queue, called Queue Underflow); if not, it reads the data at front, advances front to the next node, and — if the queue has now become empty — also resets rear to None so both pointers stay consistent.The peek() method reports the data at front without removing it, and display() walks the list from front to rear, printing every element in the order they would be dequeued. As with the stack program, a menu-driven while loop repeatedly reads the user's choice and calls the

matching method, continuing until choice 5 (Exit) is selected, which breaks the loop and ends the program.

Error Table:-

Error	*Possible Cause*	*Solution*
Queue Underflow	*dequeue() or peek() is called when the queue is empty (front is None)*	*Check if the queue is empty before dequeuing/peeking; enqueue an element first*
ValueError: invalid literal for int()	*Non-numeric text entered at the choice or element prompt*	*Enter only numeric values, or wrap input() in a try–except block*
Invalid choice printed repeatedly	*A menu option outside the range 1–5 is entered*	*Enter a valid option between 1 and 5*
rear left pointing to a stale node	*front becomes None after dequeue but rear is not reset*	*Ensure dequeue() sets rear = None whenever the last element is removed (already handled in this code)*

6(b). program to implement queue using array

Aim:-

To write a Python program to implement the operations of a Queue (Enqueue, Dequeue, Peek, and Display) using an array , and to verify its working through a menu-driven interface.

Code:-

queue using array.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/queue using a

File Edit Format Run Options Window Help

```
queue = []
front = 0
rear = -1

# Enqueue
queue.append(10)
rear = rear + 1

queue.append(20)
rear = rear + 1

queue.append(30)
rear = rear + 1

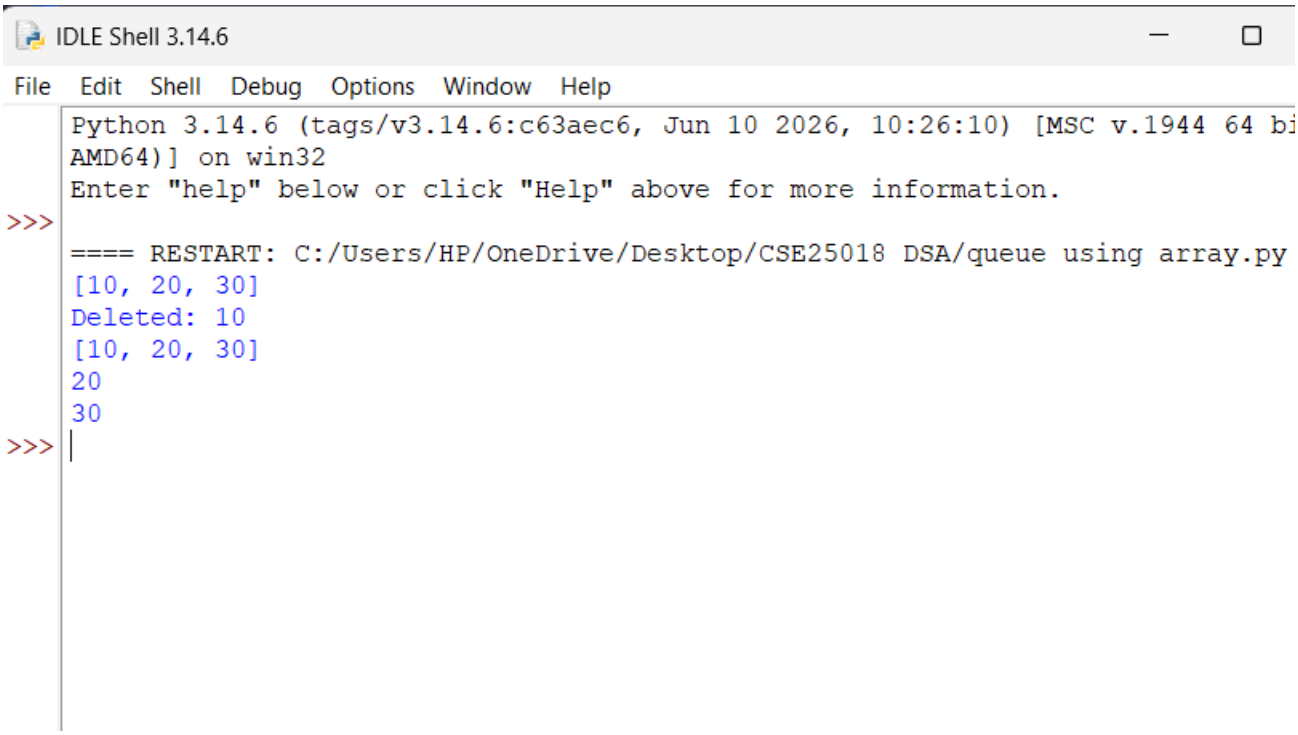
print(queue)

# Dequeue
if front > rear:
    print("Queue Underflow")
else:
    print("Deleted:", queue[front])
    front = front + 1

print(queue)

# Display
if front > rear:
    print("Queue is Empty")
else:
    for i in range(front, rear + 1):
        print(queue[i])
```

Code output:-



Explanation:

The queue using an array is implemented using a Python list along with two variables called front and rear. The front variable indicates the position from where elements are removed, while rear indicates the position where new elements are inserted. Initially, front is set to 0 and rear is set to -1, indicating that the queue is empty. During the enqueue operation, an element is added to the array and rear is increased. During the dequeue operation, the element at the front position is removed logically by moving front to the next position. The queue follows FIFO so the element inserted first is removed first.

ERROR TABLE:

Error	solution
Queue Overflow	rear == MAX - 1
Queue Underflow	front > rear
Invalid Dequeue	front > rear

LAB-7

7a) Question:

Write a Python program to implement a binary tree using an array The program should have a menu to insert a node, do inorder, preorder and postorder traversal, and display the tree.

Code:

binary tree array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-7\binary tree array.py (3.14.6)

File Edit Format Run Options Window Help

```
SIZE = 100
tree = [None] * SIZE

def insert(value):
    i = 0

    while i < SIZE:
        if tree[i] is None:
            tree[i] = value
            print("Inserted successfully")
            return

        if value < tree[i]:
            i = 2 * i + 1
        elif value > tree[i]:
            i = 2 * i + 2
        else:
            print("Duplicate value")
            return

    print("Tree is full")

def search(value):
    i = 0

    while i < SIZE:
        if tree[i] is None:
            print("Value not found")
            return

        if tree[i] == value:
            print("Value found")
            return
        elif value < tree[i]:
            i = 2 * i + 1
        else:
            i = 2 * i + 2

    print("Value not found")

def inorder(i):
    if i >= SIZE or tree[i] is None:
        return
    inorder(2 * i + 1)
    print(tree[i], end=" ")
    inorder(2 * i + 2)
```

binary tree array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-7\binary tree array.py (3.14.6)

File Edit Format Run Options Window Help

```
        return
    print(tree[i], end=" ")
    preorder(2 * i + 1)
    preorder(2 * i + 2)

def postorder(i):
    if i >= SIZE or tree[i] is None:
        return
    postorder(2 * i + 1)
    postorder(2 * i + 2)
    print(tree[i], end=" ")

while True:
    print("\n--- BST ARRAY MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Inorder")
    print("4. Preorder")
    print("5. Postorder")
    print("6. Display Array")
    print("7. Exit")

    choice = int(input("Enter choice: "))
    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)
    elif choice == 2:
        value = int(input("Enter value to search: "))
        search(value)
    elif choice == 3:
        print("Inorder:", end=" ")
        inorder(0)
        print()
    elif choice == 4:
        print("Preorder:", end=" ")
        preorder(0)
        print()
    elif choice == 5:
        print("Postorder:", end=" ")
        postorder(0)
        print()
    elif choice == 6:
        print([x for x in tree if x is not None])
    elif choice == 7:
        break
    else:
        print("Invalid choice")
```

Output:

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
5. Postorder
6. Display Array
7. Exit
Enter choice: 1
Enter value: 20
Inserted successfully

--- BST ARRAY MENU ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Display Array
7. Exit
Enter choice: 1
Enter value: 30
Inserted successfully

--- BST ARRAY MENU ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Display Array
7. Exit
Enter choice: 1
Enter value: 1
Inserted successfully

--- BST ARRAY MENU ---
1. Insert
2. Search
3. Inorder
4. Preorder
5. Postorder
6. Display Array
7. Exit
Enter choice: 3
Inorder: 1 20 30

--- BST ARRAY MENU ---
1. Insert
2. Search
3. Inorder
4. Preorder
```

Description:

This program stores the binary tree in an array with a fixed size of 100. New values are added one after another, level by level, so each value just goes to the next free place in the array.

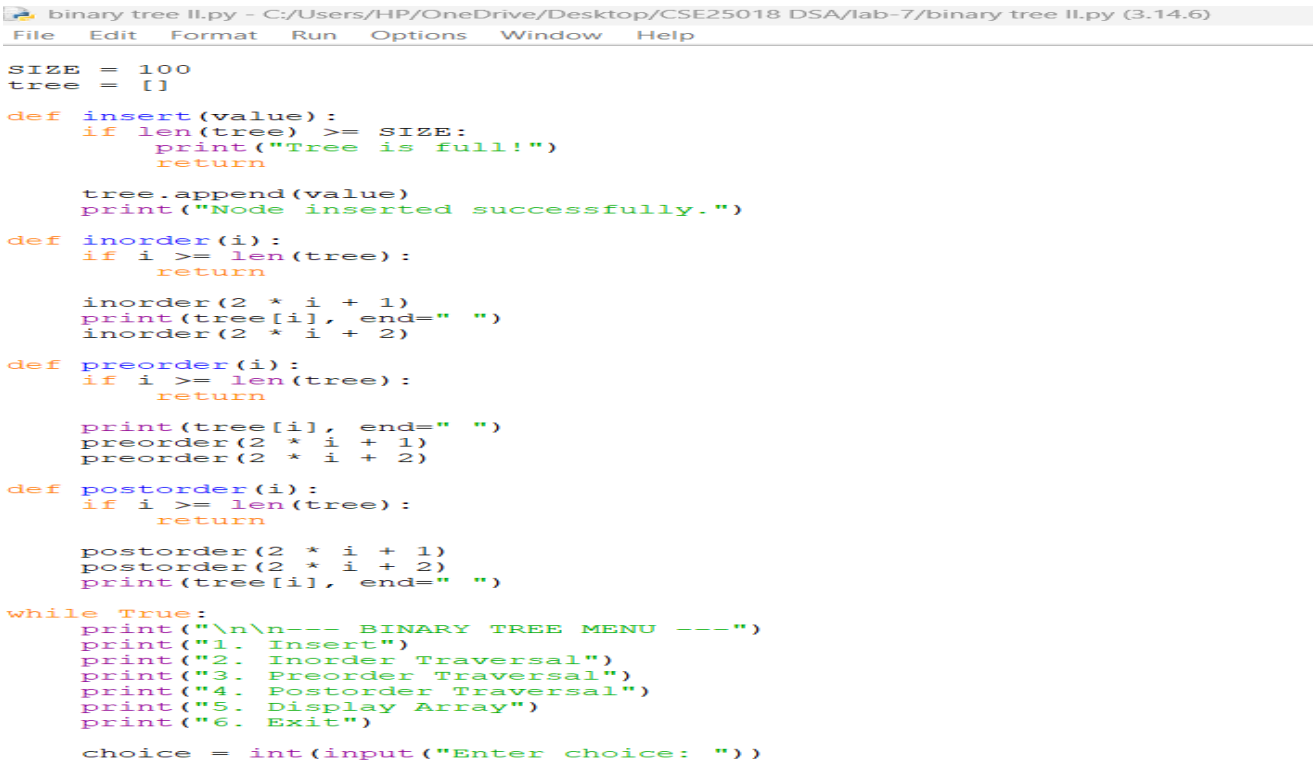
We do not need pointers here. If a node is at index i, its left child is at 2i + 1 and its right child is at 2i + 2. The position in the array is what gives the tree its shape.

The three traversals are written using recursion. Inorder goes left, root, right. Preorder goes root, left, right. Postorder goes left, right, root. Each function stops when i reaches the end of the tree. Once 100 nodes are stored, the program says 'Tree is full!' and does not add more.

Error	**Cause**	**Fix**
ValueError: invalid literal for int()	A letter or an empty value was typed when a number was expected.	Type only whole numbers, or wrap int(input()) in try/except.
IndexError: list index out of range	The line 'if i >= len(tree): return' was left out of a traversal function.	Check i >= len(tree) before using tree[i].
Tree is full!	More than 100 values were inserted into the fixed-size array.	Increase SIZE, or use the list version (7B).

7b) Write a Python program to implement a binary tree using a list The program should have a menu to insert a node, do inorder, preorder and postorder traversal, and display the tree.

Code:



binary tree II.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/lab-7/binary tree II.py (3.14.6)

File Edit Format Run Options Window Help

```
    if i >= len(tree):
        return

    postorder(2 * i + 1)
    postorder(2 * i + 2)
    print(tree[i], end=" ")

while True:
    print("\n\n--- BINARY TREE MENU ---")
    print("1. Insert")
    print("2. Inorder Traversal")
    print("3. Preorder Traversal")
    print("4. Postorder Traversal")
    print("5. Display Array")
    print("6. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        print("Inorder:", end=" ")
        inorder(0)
        print()

    elif choice == 3:
        print("Preorder:", end=" ")
        preorder(0)
        print()

    elif choice == 4:
        print("Postorder:", end=" ")
        postorder(0)
        print()

    elif choice == 5:
        print("Tree array:", tree)

    elif choice == 6:
        print("Exiting program...")
        break

    else:
        print("Invalid choice!")
```

Output:

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Enter value: 20
Node inserted successfully.


--- BINARY TREE MENU ---
1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Display Array
6. Exit
Enter choice: 1
Enter value: 1
Node inserted successfully.


--- BINARY TREE MENU ---
1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Display Array
6. Exit
Enter choice: 2
Inorder: 20 30 1


--- BINARY TREE MENU ---
1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Display Array
6. Exit
Enter choice: 3
Preorder: 30 20 1


--- BINARY TREE MENU ---
1. Insert
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Display Array
6. Exit
Enter choice: 5
Tree array: [30, 20, 1]
```

Description:

This program keeps the binary tree in a normal Python list and adds new values with append(). Since a list can grow by itself, there is no size limit and the tree never gets full.The idea is the same as before. For a node at index i, the left child is at 2i + 1 and the right child is at 2i + 2. Values are filled level by level in the order we enter them. Inorder, preorder and postorder work exactly like in Program . The only difference is that the size check is removed, because the list keeps growing as we insert.

Error	**Cause**	**Fix**
ValueError: invalid literal for int()	A letter or an empty value was typed when a number was expected.	Type only whole numbers, or wrap int(input()) in try/except.
IndexError: list index out of range	The line 'if i >= len(tree): return' was left out of a traversal function.	Check i >= len(tree) before using tree[i].
Inorder output is not sorted	A binary tree is filled level by level, not by value, so sorted output is not expected.	Use a Binary Search Tree (Lab 8) if sorted output is needed.

LAB-8

8a)Question:

Write a Python program to implement a binary search tree using an array . The program should have a menu to insert a value, search for a value, and do inorder, preorder and postorder traversal.

Code:

binary search tree array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-8\binary search tree array.py (3.14.6)

File Edit Format Run Options Window Help

```
SIZE = 100
tree = [None] * SIZE

def insert(value):
    i = 0

    while i < SIZE:
        if tree[i] is None:
            tree[i] = value
            print("Inserted successfully")
            return

        if value < tree[i]:
            i = 2 * i + 1
        elif value > tree[i]:
            i = 2 * i + 2
        else:
            print("Duplicate value")
            return

    print("Tree is full")

def search(value):
    i = 0

    while i < SIZE and tree[i] is not None:
        if tree[i] == value:
            print("Value found")
            return
        elif value < tree[i]:
            i = 2 * i + 1
        else:
            i = 2 * i + 2

    print("Value not found")

def inorder(i):
    if i >= SIZE or tree[i] is None:
        return
    inorder(2 * i + 1)
    print(tree[i], end=" ")
    inorder(2 * i + 2)

def preorder(i):
    if i >= SIZE or tree[i] is None:
        return
```

binary search tree array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-8\binary search tree array.py (3.14.6)

File Edit Format Run Options Window Help

```
    if i >= SIZE or tree[i] is None:
        return
    inorder(2 * i + 1)
    print(tree[i], end=" ")
    inorder(2 * i + 2)

def preorder(i):
    if i >= SIZE or tree[i] is None:
        return
    print(tree[i], end=" ")
    preorder(2 * i + 1)
    preorder(2 * i + 2)

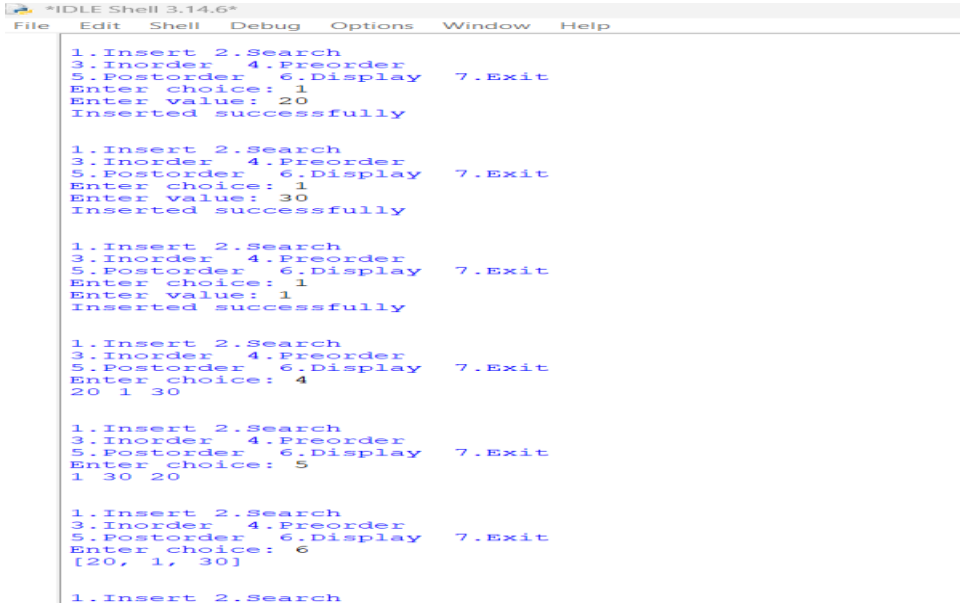
def postorder(i):
    if i >= SIZE or tree[i] is None:
        return
    postorder(2 * i + 1)
    postorder(2 * i + 2)
    print(tree[i], end=" ")

while True:
    print("\n\n1.Insert 2.Search")
    print("3.Inorder  4.Preorder")
    print("5.Postorder  6.Display  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        insert(int(input("Enter value: ")))
    elif choice == 2:
        search(int(input("Enter value: ")))
    elif choice == 3:
        inorder(0)
        print()
    elif choice == 4:
        preorder(0)
        print()
    elif choice == 5:
        postorder(0)
        print()
    elif choice == 6:
        print([x for x in tree if x is not None])
    elif choice == 7:
        break
    else:
        print("Invalid choice")
```

Output:



Description:

Here the BST is kept in an array of size 100. Empty places hold None. Every insertion starts from index 0, which is the root.

At each node we compare the new value with the value stored there. If it is smaller we go to the left child (2i + 1), and if it is bigger we go to the right child (2i + 2). We keep going until we find an empty place and put the value there. If the value is already present, the program prints 'Duplicate value'.

Search follows the same path and tells us 'Value found' or 'Value not found'. The traversals simply skip the empty places, and inorder gives the numbers in sorted order. One problem with this method is that if the values are entered in increasing order, the tree leans to one side and the indexes grow very fast, so 'Tree is full' can show up even with only a few nodes.

Error	**Cause**	**Fix**
Duplicate value	The value we are inserting is already in the tree.	Insert a different value, since this BST keeps only unique values.
Tree is full	The path went past index 99 of the array, which happens easily in a one-sided tree.	Increase SIZE, or use the linked-list version (8B).
ValueError: invalid literal for int()	A letter or an empty value was typed when a number was expected.	Type only whole numbers, or wrap int(input()) in try/except.

8b) Write a Python program to implement a binary search tree using a linked list The program should have a menu to insert a value, search for a value, and do inorder, preorder and postorder traversal.

Code:

*binary search tree linked list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-8\binary search

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, value):
        self.data = value
        self.left = None
        self.right = None

root = None

def insert(node, value):
    if node is None:
        return Node(value)

    if value < node.data:
        node.left = insert(node.left, value)
    elif value > node.data:
        node.right = insert(node.right, value)
    else:
        print("Duplicate value not allowed")

    return node

def search(node, value):
    if node is None:
        print("Value not found")
    elif node.data == value:
        print("Value found")
    elif value < node.data:
        search(node.left, value)
    else:
        search(node.right, value)

def inorder(node):
    if node:
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)

def preorder(node):
    if node:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
```

binary search tree linked list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-8\binary search tree linked list.py (3.14.6)

File Edit Format Run Options Window Help

```
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)

def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")

while True:
    print("\n--- BST LINKED LIST MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Inorder")
    print("4. Preorder")
    print("5. Postorder")
    print("6. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        root = insert(root, value)
        print("Insertion operation completed")

    elif choice == 2:
        value = int(input("Enter value to search: "))
        search(root, value)

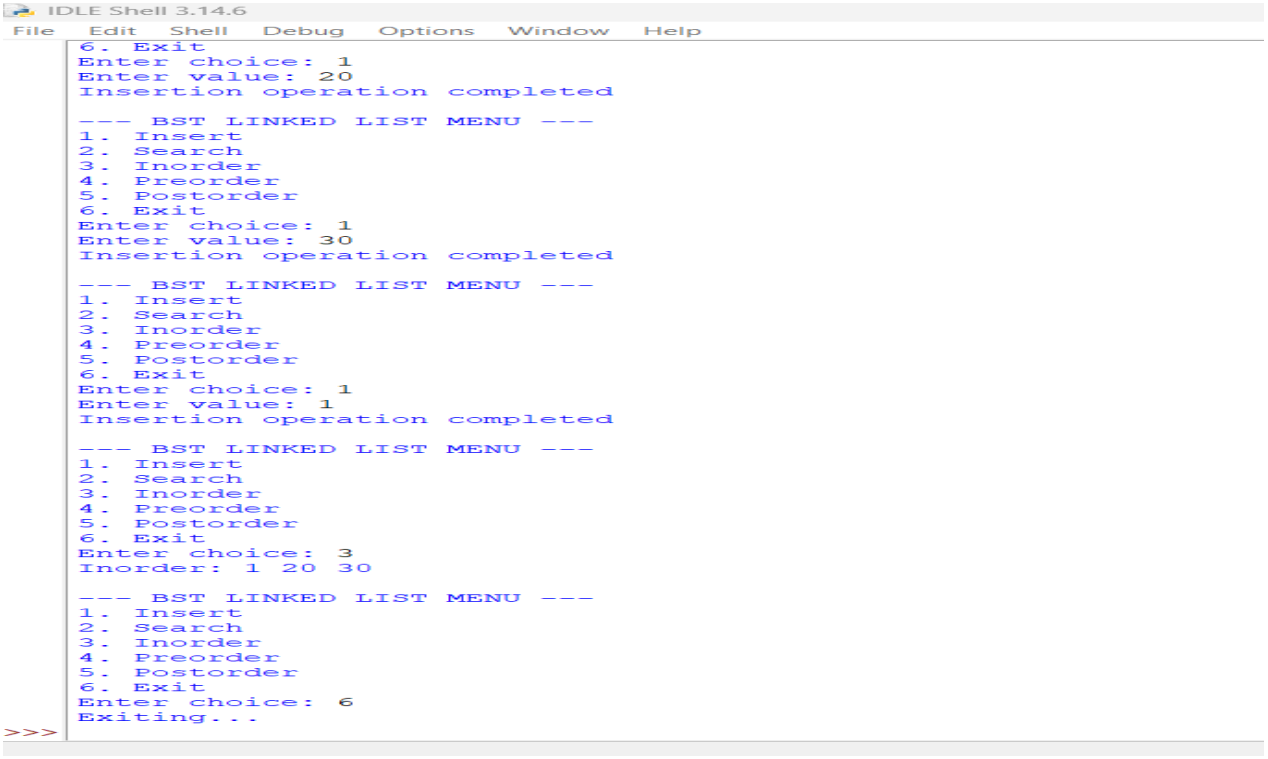
    elif choice == 3:
        print("Inorder:", end=" ")
        inorder(root)
        print()

    elif choice == 4:
        print("Preorder:", end=" ")
        preorder(root)
        print()

    elif choice == 5:
        print("Postorder:", end=" ")
        postorder(root)
        print()

    elif choice == 6:
        print("Exiting...")
```

Output:



Description:

In this version every node is an object of a Node class that has three parts: data, left and right. The variable root points to the first node, and it stays None while the tree is empty.The insert function is recursive. It goes left for a smaller value, goes right for a bigger value, and makes a new Node when it reaches an empty spot. A repeated value gives the message 'Duplicate value not allowed'. The function returns the node so the parent can link to it, which is why we write root = insert(root, value).Search is also recursive and prints whether the value was found. Inorder traversal prints the values in sorted order. There is no size limit, because memory is used only for the nodes we actually create.

Error	**Cause**	**Fix**
Duplicate value not allowed	The value we are inserting is already in the tree.	Insert a different value, since this BST keeps only unique values.
Tree stays empty	The result of insert() is not saved back in root.	Call it as root = insert(root, value).
AttributeError: 'NoneType' object has no attribute 'left'	insert() does not return node, so the parent gets None.	Add 'return node' at the end of insert().

Lab-9

9a) Question:

Write a Python program to implement an AVL tree using an array . The program should have a menu to insert a value with automatic balancing (rotations), search for a value, and do inorder, preorder and postorder traversal.

Code:

avl trees array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees array.py (3.14.6)

File Edit Format Run Options Window Help

```
SIZE = 100
key = [0] * SIZE
left = [-1] * SIZE
right = [-1] * SIZE
height = [0] * SIZE
count = 0
root = -1

def h(i):
    return height[i] if i != -1 else 0

def new_node(value):
    global count
    i = count
    count += 1
    key[i] = value
    height[i] = 1
    return i

def update(i):
    height[i] = 1 + max(h(left[i]), h(right[i]))

def balance(i):
    return h(left[i]) - h(right[i])

def rotate_right(y):
    x = left[y]
    t = right[x]
    right[x] = y
    left[y] = t
    update(y)
    update(x)
    return x

def rotate_left(x):
    y = right[x]
    t = left[y]
    left[y] = x
    right[x] = t
    update(x)
    update(y)
    return y

def insert(node, value):
    if node == -1:
        if count >= SIZE:
            print("Tree is full")
```

avl trees array.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees array.py (3.14.6)

File Edit Format Run Options Window Help

```
            if count >= SIZE:
                print("Tree is full")
                return -1
            return new_node(value)

        if value < key[node]:
            child = insert(left[node], value)
            if child != -1 or left[node] == -1:
                left[node] = child
        elif value > key[node]:
            child = insert(right[node], value)
            if child != -1 or right[node] == -1:
                right[node] = child
        else:
            print("Duplicate value")
            return node

        update(node)
        b = balance(node)

        if b > 1 and value < key[left[node]]:
            return rotate_right(node)

        if b < -1 and value > key[right[node]]:
            return rotate_left(node)

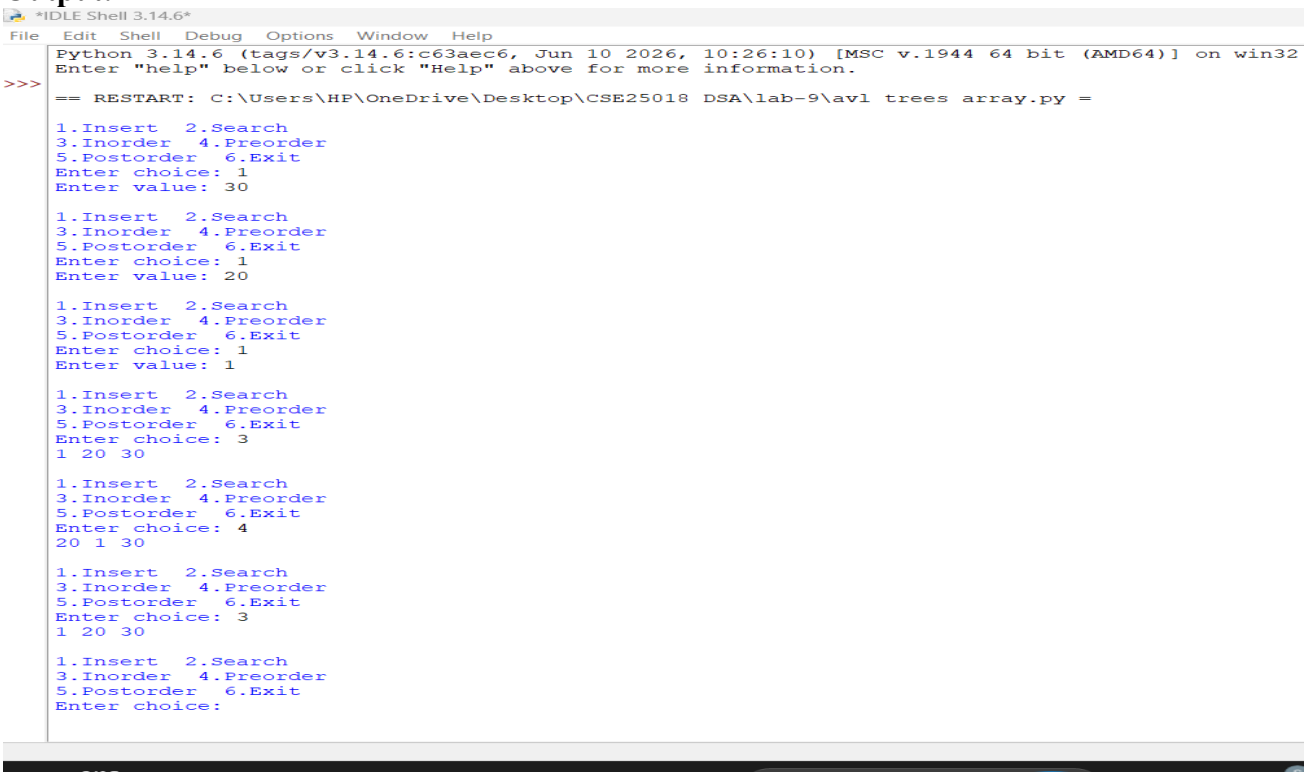
        if b > 1 and value > key[left[node]]:
            left[node] = rotate_left(left[node])
            return rotate_right(node)

        if b < -1 and value < key[right[node]]:
            right[node] = rotate_right(right[node])
            return rotate_left(node)

        return node

def search(value):
    i = root
    while i != -1:
        if key[i] == value:
            print("Value found")
            return
        if value < key[i]:
            i = left[i]
        else:
            i = right[i]
    print("Value not found")
```

Output:



Description: An AVL tree is a binary search tree that balances itself. For every node, the balance factor (left height minus right height) must be -1, 0 or 1. Because of this the tree stays short, and insert and search take O(log n) time.In this program the tree is stored in four arrays of size 100: key for the value, left and right for the child indexes (-1 means there is no child), and height. The variable count tells us the next free place, and root keeps the index of the root node.After each insertion the heights are updated and the balance factor is checked. If it goes above 1 or below -1, one of four cases is fixed. Left-Left uses a right rotation. Right-Right uses a left rotation. Left-Right needs a left rotation on the left child and then a right rotation. Right-Left needs a right rotation on the right child and then a left rotation.

Error	**Cause**	**Fix**
Duplicate value	The same value was inserted a second time.	Insert only values that are not already in the tree.
Tree is full	count reached SIZE (100), so there is no free place for a new node.	Increase SIZE and run the program again.
Tree not balanced	The height was not updated after insertion, so the balance factor is wrong.	Call update(node) after inserting, before checking balance(node).

9b)

Write a Python program to implement an AVL tree using a list. The program should have a menu to insert a value with automatic balancing (rotations), search for a value, and do inorder, preorder and postorder traversal.

Code:

avl trees list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees list.py (3.14.6)

File Edit Format Run Options Window Help

```
SIZE = 100
key = [0] * SIZE
left = [-1] * SIZE
right = [-1] * SIZE
height = [0] * SIZE
count = 0
root = -1

def h(i):
    return height[i] if i != -1 else 0

def new_node(value):
    global count
    i = count
    count += 1
    key[i] = value
    height[i] = 1
    return i

def update(i):
    height[i] = 1 + max(h(left[i]), h(right[i]))

def balance(i):
    return h(left[i]) - h(right[i])

def rotate_right(y):
    x = left[y]
    t = right[x]
    right[x] = y
    left[y] = t
    update(y)
    update(x)
    return x

def rotate_left(x):
    y = right[x]
    t = left[y]
    left[y] = x
    right[x] = t
    update(x)
    update(y)
    return y

def insert(node, value):
    if node == -1:
        if count >= SIZE:
            print("Tree is full")
```

avl trees list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees list.py (3.14.6)

File Edit Format Run Options Window Help

```
    update(y)
    return y

def insert(node, value):
    if node == -1:
        if count >= SIZE:
            print("Tree is full")
            return -1
        return new_node(value)

    if value < key[node]:
        child = insert(left[node], value)
        if child != -1 or left[node] == -1:
            left[node] = child
    elif value > key[node]:
        child = insert(right[node], value)
        if child != -1 or right[node] == -1:
            right[node] = child
    else:
        print("Duplicate value")
        return node

    update(node)
    b = balance(node)

    if b > 1 and value < key[left[node]]:
        return rotate_right(node)

    if b < -1 and value > key[right[node]]:
        return rotate_left(node)

    if b > 1 and value > key[left[node]]:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if b < -1 and value < key[right[node]]:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node

def search(value):
    i = root
    while i != -1:
        if key[i] == value:
            print("Value found")
            return
```

avl trees list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees list.py (3.14.6)

File Edit Format Run Options Window Help

```
            i = right[i]
    print("Value not found")

def inorder(i):
    if i == -1:
        return
    inorder(left[i])
    print(key[i], end=" ")
    inorder(right[i])

def preorder(i):
    if i == -1:
        return
    print(key[i], end=" ")
    preorder(left[i])
    preorder(right[i])

def postorder(i):
    if i == -1:
        return
    postorder(left[i])
    postorder(right[i])
    print(key[i], end=" ")

while True:
    print("\n1.Insert  2.Search")
    print("3.Inorder  4.Preorder")
    print("5.Postorder  6.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        old_count = count
        root = insert(root, value)
    elif choice == 2:
        search(int(input("Enter value: ")))
    elif choice == 3:
        inorder(root)
        print()
    elif choice == 4:
        preorder(root)
        print()
    elif choice == 5:
        postorder(root)
        print()
    elif choice == 6:
```

avl trees list.py - C:\Users\HP\OneDrive\Desktop\CSE25018 DSA\lab-9\avl trees list.py (3.14.6)

File Edit Format Run Options Window Help

```
    if i == -1:
        return
    inorder(left[i])
    print(key[i], end=" ")
    inorder(right[i])

def preorder(i):
    if i == -1:
        return
    print(key[i], end=" ")
    preorder(left[i])
    preorder(right[i])

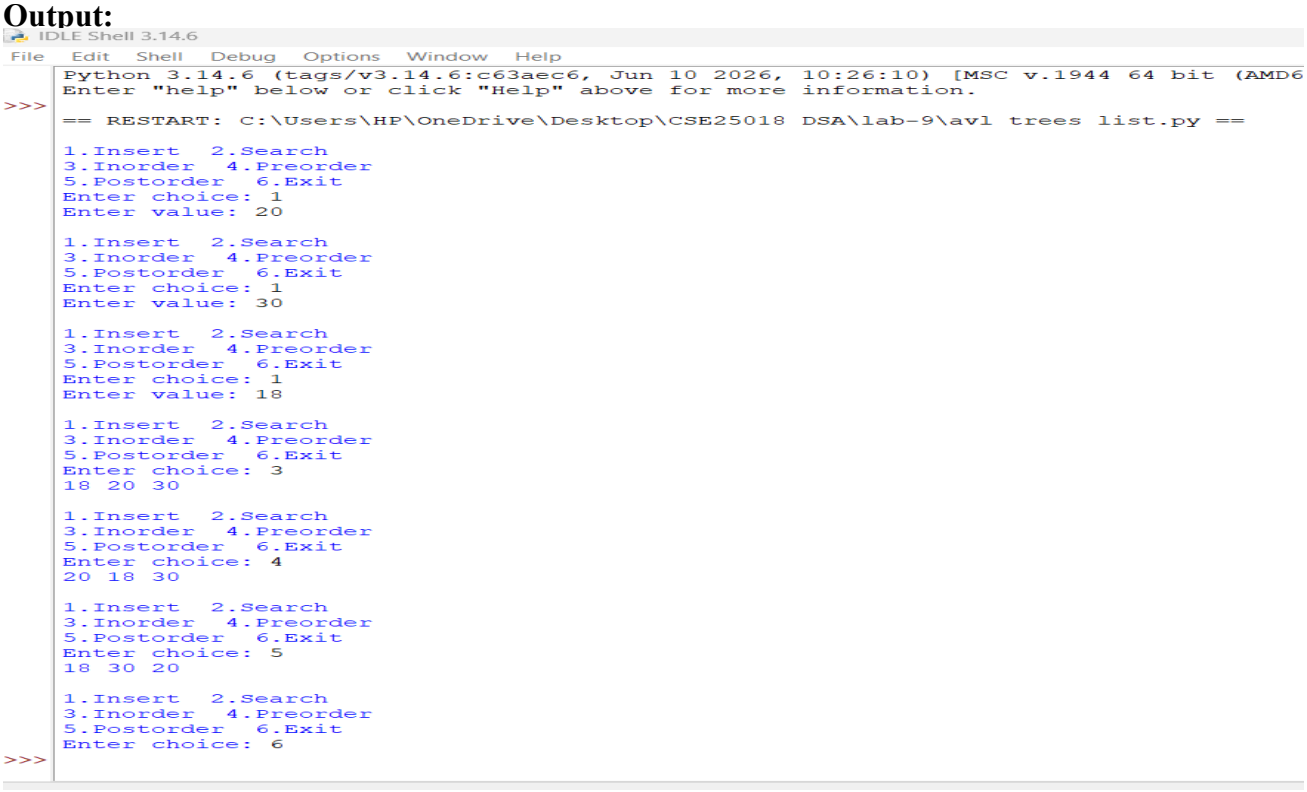
def postorder(i):
    if i == -1:
        return
    postorder(left[i])
    postorder(right[i])
    print(key[i], end=" ")

while True:
    print("\n1.Insert  2.Search")
    print("3.Inorder  4.Preorder")
    print("5.Postorder  6.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        old_count = count
        root = insert(root, value)
    elif choice == 2:
        search(int(input("Enter value: ")))
    elif choice == 3:
        inorder(root)
        print()
    elif choice == 4:
        preorder(root)
        print()
    elif choice == 5:
        postorder(root)
        print()
    elif choice == 6:
        break
    else:
        print("Invalid choice")
```

Output:



Description: This version also stores the nodes in Python lists called key, left, right and height. The index of an element works like a pointer, and -1 means there is no child. Every new node takes the next free index given by count.The working is the same as Program 9A. The insert function is recursive. It updates the height, finds the balance factor, and does a single or double rotation if the tree becomes unbalanced. Each rotation returns the new root of that part of the tree.Search starts at the root and moves left or right using the indexes. The traversals are recursive and stop when the index is -1. Inorder traversal always prints the keys in sorted order.**Error table :**

Error	**Cause**	**Fix**
Duplicate value	The same value was inserted a second time.	Insert only values that are not already in the tree.
IndexError: list index out of range	More than 100 nodes were added and the count >= SIZE check is missing.	Keep the check 'if count >= SIZE' before creating a new node.
Wrong output after rotation	The new root of the rotated subtree was not returned.	Use return rotate_left(...) or return rotate_right(...) in insert().

Lab-10

10)

Write a Python program to implement hashing using separate chaining, linear probing, and quadratic probing.

CODE:

lab-10.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/lab-10.py (3.14.6)

File Edit Format Run Options Window Help

```
size = 10

# Separate Chaining
chain = [[] for i in range(size)]

# Linear Probing
linear = [None] * size

# Quadratic Probing
quadratic = [None] * size


def separate_chaining(key):
    index = key % size
    chain[index].append(key)


def linear_probing(key):
    index = key % size

    for i in range(size):
        new_index = (index + i) % size

        if linear[new_index] is None:
            linear[new_index] = key
            return

    print("Hash table is full")


def quadratic_probing(key):
    index = key % size

    for i in range(size):
        new_index = (index + i * i) % size

        if quadratic[new_index] is None:
            quadratic[new_index] = key
            return

    print("No empty position found")


def display():
    print("\nSeparate Chaining:")
    for i in range(size):
```

lab-10.py - C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/lab-10.py (3.14.6)

File Edit Format Run Options Window Help

```
    for i in range(size):
        new_index = (index + i * i) % size

        if quadratic[new_index] is None:
            quadratic[new_index] = key
            return

    print("No empty position found")


def display():
    print("\nSeparate Chaining:")
    for i in range(size):
        print(i, ":", chain[i])

    print("\nLinear Probing:")
    for i in range(size):
        print(i, ":", linear[i])

    print("\nQuadratic Probing:")
    for i in range(size):
        print(i, ":", quadratic[i])


while True:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter the key: "))

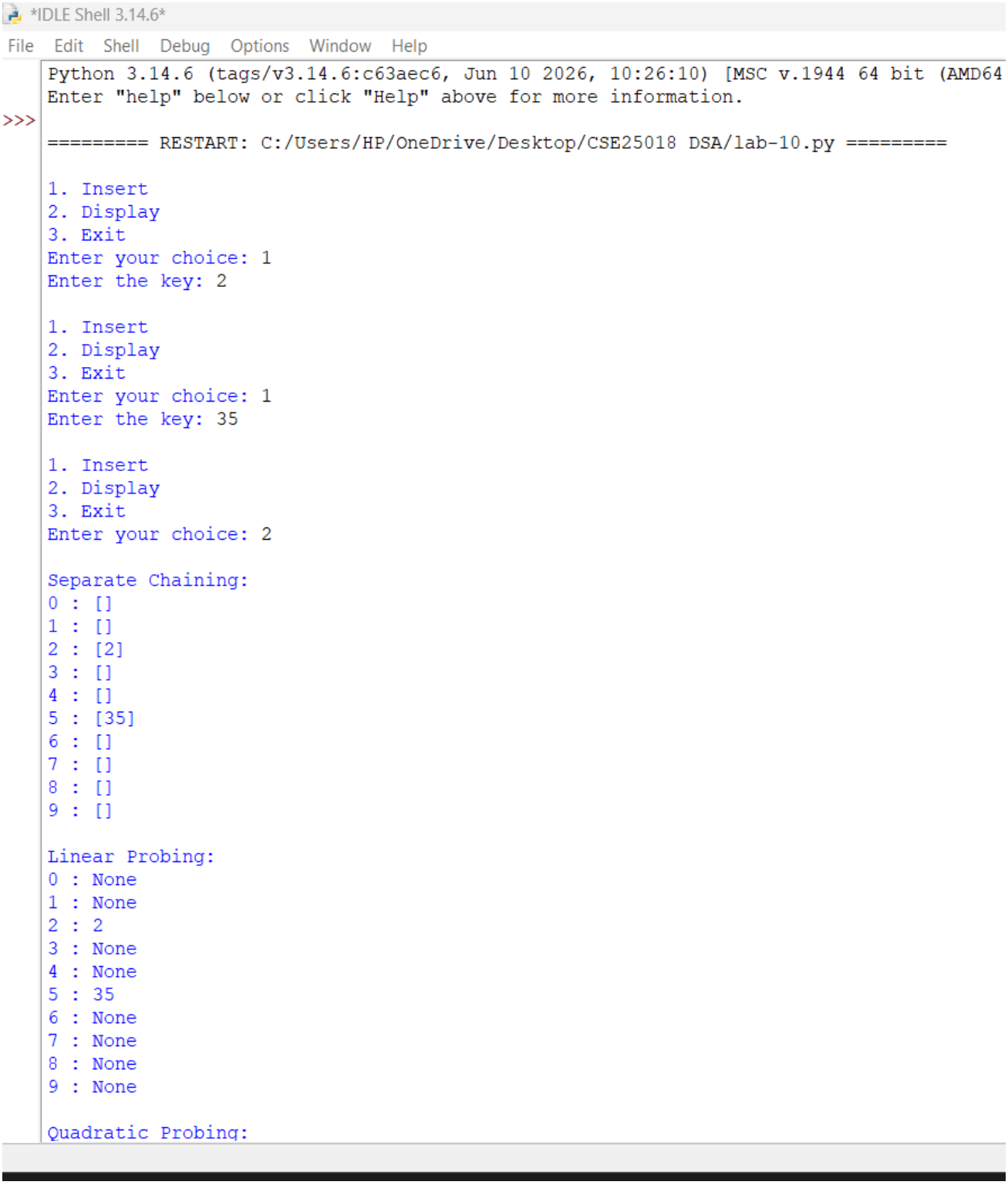
        separate_chaining(key)
        linear_probing(key)
        quadratic_probing(key)

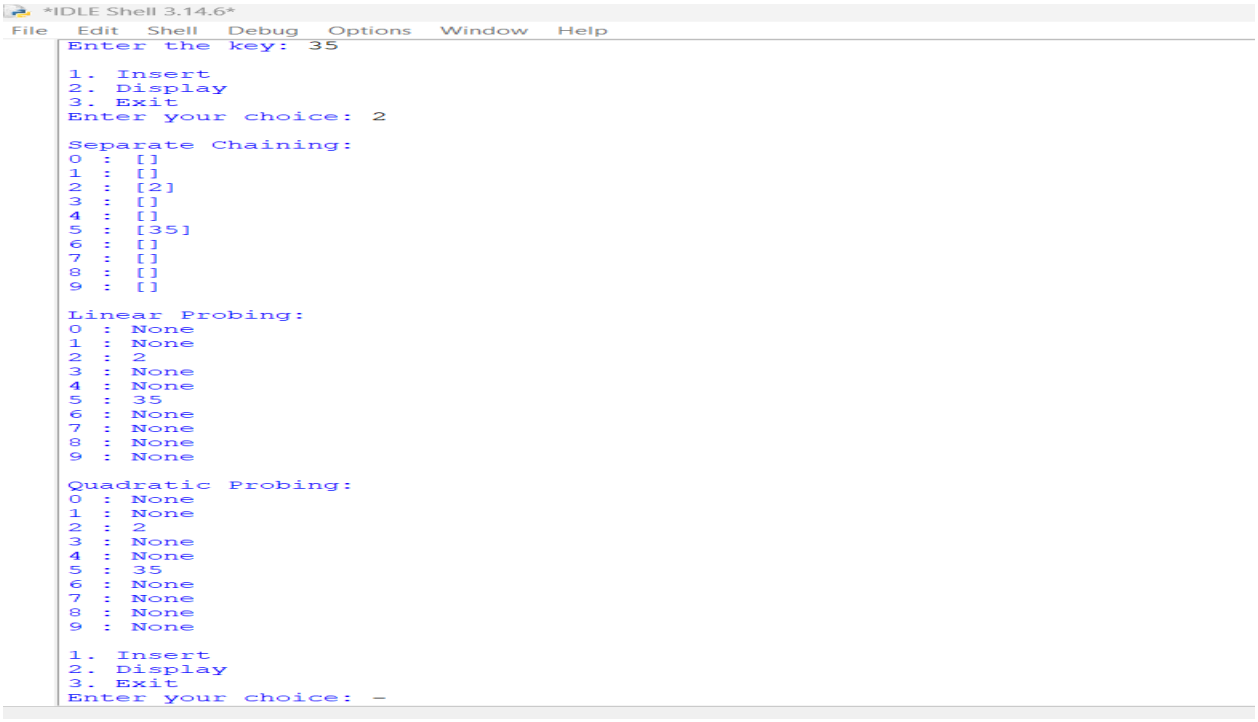
    elif choice == 2:
        display()

    elif choice == 3:
        break

    else:
        print("Invalid choice")
```

Code output:


IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64
Enter "help" below or click "Help" above for more information.
>>>
========= RESTART: C:/Users/HP/OneDrive/Desktop/CSE25018 DSA/lab-10.py =========
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter the key: 2
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter the key: 35
1. Insert
2. Display
3. Exit
Enter your choice: 2
Separate Chaining:
0 : []
1 : []
2 : [2]
3 : []
4 : []
5 : [35]
6 : []
7 : []
8 : []
9 : []
Linear Probing:
0 : None
1 : None
2 : 2
3 : None
4 : None
5 : 35
6 : None
7 : None
8 : None
9 : None
Quadratic Probing:



Description:

This program implements hashing using three collision resolution techniques: Separate Chaining, Linear Probing, and Quadratic Probing. It uses a hash function key % size to calculate the index for each key. In Separate Chaining, multiple keys with the same hash index are stored in a list. In Linear Probing, the next available position is checked sequentially when a collision occurs. In Quadratic Probing, the next available position is found using the square of the probe number. The program allows the user to insert keys, display all three hash tables, and exit using a menu-driven approach.

Error table:

S.No	Error	Reason
1	ValueError	Entering a non-numeric value instead of an integer.
2	Hash table is full	No empty position is available for insertion.